

학원 운영 관리 프로그램

1차 기획서 원판

A07팀

202111316 안성민

202311332 이담

202411358 이지애

202513246 우민재

202513284 이승은

학원의 반 편성과 수강 등록을 관리하는 프로그램.

메뉴 번호를 키 입력하는 방식으로 사용하며, 사용자가 메모장 등으로 직접 편집할 수도 있는 텍스트 파일에 데이터를 저장합니다. 원장, 강사, 학생 세 역할로 나뉘어 로그인하고 역할에 따라 사용할 수 있는 기능이 다르며, 한 강사가 같은 시간에 두 반을 맡거나 한 학생이 같은 시간에 두 반을 수강하는

등의 모순을 검사합니다. 실제 컴퓨터 시각 대신 사용자가 지정하는 가상의 현재 일시를 사용하고, 이 일시는 저장된 기록보다 과거로 되돌릴 수 없습니다.

목차

1. 용어	5
2. 기본 사항	10
2.1 작동 환경	10
2.2 프로그램 구성	10
2.3 프로그램 설치 및 실행	11
2.3.1 최초 실행	12
2.4 사용자 역할과 권한	12
2.5 프로젝트 범위	13
3. 프로그램 사용 흐름도	14
3.1 학생	17
3.1.1 로그인	17
3.1.2 학생 메뉴 선택	17
3.1.3 내 정보 조회	18
3.1.4 수강 가능한 반 조회	18
3.1.5 반 상세 조회	18
3.1.6 수강 신청	18
3.1.7 내 수강 목록 조회	19
3.1.8 수강 취소	19
3.1.9 로그아웃	19
3.2 강사	19
3.2.1 로그인	20
3.2.2 강사 메뉴 선택	20
3.2.3 담당 반 조회	20
3.2.4 수업 일정 조회	20
3.2.5 담당 반 수강생 조회	21
3.2.6 로그아웃	21
3.3 원장	21
3.3.1 로그인	21
3.3.2 원장 메뉴 선택	22
3.3.3 학생 관리	22
3.3.4 강사 관리	22
3.3.5 과목 관리	22
3.3.6 반 관리	23
3.3.7 수강 등록 조회·취소	23
3.3.8 가상 현재 일시 변경	23
3.3.9 무결성 검사	23

3.3.10 로그아웃	23
4. 데이터 요소	24
4.1 식별자	24
4.1.1 사용자 ID	24
4.1.2 내부 식별자	25
4.2 이름	28
4.2.1 사람 이름	28
4.2.2 과목명	29
4.2.3 반 이름	30
4.3 비밀번호	31
4.4 사용자 역할	32
4.5 연락처	32
4.6 상태	33
4.6.1 학생 상태	34
4.6.2 강사 상태	34
4.6.3 반 상태	35
4.6.4 수강 등록 상태	36
4.7 날짜	36
4.8 시각	37
4.9 일시	38
4.10 교시	38
4.11 요일	40
4.11.1 요일 목록	41
4.12 수업 일정	42
4.12.1 일정 충돌	42
4.13 정원	44
4.14 가상 현재 일시	45
4.15 수강 등록	49
5. 데이터 파일	51
5.1 공통 규칙	52
5.1.1 인코딩	52
5.1.2 헤더	52
5.1.3 레코드	53
5.1.4 필드 구분	53
5.1.5 필드의 개수	53
5.1.6 빈 줄	53
5.1.7 공백	54
5.1.8 빈 값	54
5.1.9 주석	54
5.1.10 마지막 개행	54
5.1.11 레코드 순서	55
5.1.12 자동 수정	55
5.1.13 내부 식별자의 부여	55
5.1.14 문법 오류와 의미 오류	56

5.2 파일 형식	56
5.2.1 사용자 계정 파일	56
5.2.2 학생 파일	58
5.2.3 강사 파일	59
5.2.4 과목 파일	60
5.2.5 반 파일	61
5.2.6 수강 등록 파일	63
5.2.7 가상 현재 일시 파일	64
5.3 부가 확인 항목	65
5.4 무결성 확인 및 처리	66
6. 주 프롬프트	69
6.1 메인 메뉴	69
6.1.1 역할별 메인메뉴 화면	69
6.1.2 메뉴 입력 오류	71
6.1.3 확인(Y/N) 프롬프트	71
6.1.4 프로그램 종료	71
6.1.5 빈 입력 및 입력 중 취소	72
6.1.6 조회 결과 정렬	72
6.2 로그인	73
6.2.1 로그인 화면	73
6.2.2 인증 실패	73
6.2.3 연속 실패 처리	74
6.2.4 로그인 성공	74
6.2.5 로그아웃	75
6.2.6 로그인 보안 범위	75
6.3 가상 현재 일시	75
6.3.1 현재 값 표시 및 입력	75
6.3.2 형식·의미 오류	76
6.3.3 시각 역행	76
6.3.4 변경 확인	76
6.3.5 저장 결과	76
6.4 학생 메뉴	77
6.4.1 내 정보 조회	77
6.4.2 수강 가능한 반 조회	77
6.4.3 반 상세 정보 조회	78
6.4.4 수강 신청	79
6.4.5 내 수강 목록 조회	80
6.4.6 수강 취소	81
6.4.7 로그아웃	81
6.5 강사 메뉴	82
6.5.1 담당 반 조회	82
6.5.2 담당 반 수강생 조회	82
6.5.3 수업 일정 조회	84
6.5.4 로그아웃	85

6.6 원장 메뉴	85
6.6.1 학생 관리	85
6.6.1.1 목록 조회	86
6.6.1.2 상세 조회	86
6.6.1.3 학생 검색	86
6.6.1.4 등록	87
6.6.1.5 수정	87
6.6.1.6 삭제	88
6.6.2 강사 관리	88
6.6.2.1 목록 조회	89
6.6.2.2 상세 조회	89
6.6.2.3 강사 검색	90
6.6.2.4 등록	90
6.6.2.5 수정	91
6.6.2.6 삭제	91
6.6.3 과목 관리	92
6.6.3.1 목록 조회	92
6.6.3.2 상세 조회	92
6.6.3.3 과목 검색	93
6.6.3.4 등록	93
6.6.3.5 수정	93
6.6.3.6 삭제	93
6.6.4 반 관리	94
6.6.4.1 목록 조회	94
6.6.4.2 상세 조회	94
6.6.4.3 개설	95
6.6.4.4 수정	96
6.6.4.5 폐강	97
6.6.5 수강 등록 조회·취소	97
6.6.5.1 전체 수강 등록 조회	97
6.6.5.2 반별 수강 등록 조회	98
6.6.5.3 학생별 수강 등록 조회	98
6.6.5.4 수강 등록 취소	98
6.6.6 무결성 검사	99
6.6.7 로그아웃	100
6.6.8 계정 비밀번호 관리	100
6.7 종료	101
6.7.1 정상 종료 — 메뉴에서 선택	101
6.7.2 로그인 연속 실패로 인한 종료	102
6.7.3 입력 도중 중단 (Ctrl+C, EOF)	102
6.7.4 시작 시 치명적 오류로 인한 종료	102
6.7.5 종료 시 공통 사항	103
7. 기타 처리	103
7.1 구현 완료 조건	103

1. 용어

이 문서에서 사용할 용어들의 의미를 약속해둔다.

각 용어 뒤에 붙은 작은 표식은 그 용어를 어떤 성격으로 정하는지를 나타내며, 다음과 같이 분류된다.

기준 — 이미 세간에서 지배적으로 통용되는 의미대로지만, 소개나 설명이 필요한 경우

특정 — 세간에서 통용되는 의미에서 벗어나지는 않고 여전히 그에 포함되지만, 보다 특정한 범위나 대상으로 좁혀서 사용하려는 경우

명확 — 세간에서 둘 이상의 의미들로 혼란스럽게 통용되고 있어서, 그중 어떤 의미로 사용할지 명확히 정하려는 경우

변경 — 세간에서 널리 통용되는 의미에서 다소 벗어난 다른 의미로 바꿔 사용하려는 경우

신규 — 세간에서는 거의 사용되고 있지 않은 용어를 이 문서에서만 쓰려고 새로 정의하는 경우

아래 용어들은 내용상 관련된 것끼리 가까이 묶되, 어떤 용어 A를 이용해서 다른 용어 B를 정의해야 하는 경우 A를 B보다 먼저 정의하고 있다. 따라서 이 목록은 가나다순이 아니며, 위에서부터 순서대로 읽는 것을 전제한다.

학원 관리 프로그램 (혹은 **프로그램**)^{특정} 이 문서를 통해 기획·명세하고 있는 대상 프로그램인 《학원 운영 관리 프로그램》.

실행 파일 (혹은 **실행 파일**)^{특정+변경} 학원 관리 프로그램을 실행시키기 위해 꼭 필요한 본체에 해당하는 파일. 엄밀히 이 파일은 스스로 실행 가능한 파일이 아니라 Python 인터프리터에 의해 읽히고 해석될 내용을 담고 있는 텍스트 형식의 파일이지만, 이 문서에서는 혼동될 만한 다른 실행 파일을 언급할 일이 없기 때문에 편의상 그냥 이 파일을 실행 파일이라고 부른다.

데이터 파일^{특정} 학원 관리 프로그램이 학원 운영 정보를 저장하고 읽어 들이는 텍스트 형식의 파일. 이 프로그램은 여러 개의 데이터 파일을 다루므로, 특정 파일 하나를 가리킬 때에는 언제나 users.txt와 같이 파일 이름을 함께 적는다.

숫자^{특정} (서)아라비아 숫자들 중 표준 키보드로 직접 입력할 수 있는 열 개(U+0030 '0' ~ U+0039 '9')만을 뜻한다. 따라서 이 문서에서 말하는 숫자에는 로마 숫자(V)나 각 언어별 고유 숫자 기호가 포함되지 않으며, 아라비아 숫자들 중에서도 전각 숫자(5), 원이나 괄호로 둘러싸인 숫자(①), 위·아래 첨자용 숫자 등은 모두 제외된다.

표준공백 (혹은 **공백**)^{명확} 표준 키보드의 스페이스 바로 직접 입력할 수 있는 문자(U+0020). 이 문서에서 표준공백을 시각적으로 명확히 드러내야 할 때에는 그 자리에 '␣' 기호를 써서 표시한다. 탭 문자(U+0009)는 표준공백이 아니다. 탭 문자는 데이터 파일에서 필드를 구분하는 구분자로만 사용하며, 개별 필드의 값 내부에는 포함할 수 없다. 사용자가 콘솔에서 입력하는 개별 데이터 값에도 탭 문자를 허용하지 않는다.

개행^{특정} 텍스트 형식의 파일을 편집할 때 표준 키보드의 Enter 키로 입력할 수 있는 두 문자(U+000A Line Feed, U+000D Carriage Return) 중 하나 혹은 이들의 조합. 어느 문자나 조합이 입력될지는 운영체제와 편집기마다 다르지만, 이 프로그램은 세 경우를 모두 똑같이 개행으로 취급한다. 따라서 데이터 파일을 어느 운영체제에서 작성하고 저장해도 상관없다.

행 (혹은 **줄**)^{변경} 데이터 파일 속에 등장하는, 아래 두 경우 중 하나에 해당하는 문자열.

파일의 맨 처음이나 어떤 개행 직후부터, 이후 처음 만나는 개행 직전까지.

파일의 맨 처음이나 어떤 개행 직후에 개행이 아닌 문자가 하나 이상 있고, 그대로 파일 끝에 다다른 경우 그 파일 끝까지.

위 정의에 따르면 개행 문자는 행을 구성하는 문자로 포함되지 않으며 행과 행 사이의 구분자로만 취급된다. 이는 행의 통상적인 정의와 다르니 이 문서를 읽는 동안 주의하기 바란다. 또한 첫 번째 경우에는 아무 문자도 없는 '빈 행'이 가능한 반면, 두 번째 경우에는 빈 행이 불가능하다.

길이^{명확} 문자열 속에 들어 있는 유니코드 문자들의 실제 개수. 각 글자가 메모리 상에서 차지하는 용량(바이트 수)이나 화면 상의 시각적인 폭과는 전혀 무관하다. 예를 들어 "안녕하세요"와 "Hello"의 길이는 똑같이 5이다.

필드^{특정} 하나의 레코드를 이루는 개별 값. 각 필드는 무엇을 담는 자리인지가 미리 정해져 있고, 그 자리에 올 수 있는 값의 형식도 정해져 있다.

레코드^{특정} 하나의 대상에 관한 정보를 담은 필드들의 묶음. 이 프로그램에서 한 개의 레코드는 언제나 한 개의 행에 저장되므로, 행과 레코드는 일대일로 대응한다. 따라서 하나의 레코드가 두 행에 걸쳐 저장되거나 한 행에 두 레코드가 저장되는 일은 없다.

동치^{명확} 두 값을 같은 값으로 볼지 판단한 결과가 참인 상태. 무엇을 같은 값으로 볼지는 값의 종류마다 따로 정하며, 특히 대소문자를 구별하는지와 앞뒤 공백을 무시하는지를 반드시 명시한다. 예를 들어 학생 번호를 대소문자까지 구별해 비교하기로 정했다면 S0001과 s0001은 서로 동치가 아니다.

표준형^{신규} 서로 동치인 여러 표기들 중, 프로그램이 데이터 파일에 저장할 때 사용하기로 정한 하나의 표기. 사용자가 다른 표기로 입력하더라도 프로그램은 그 값을 표준형으로 바꾸어 저장한다. 예를 들어 연락처의 표준형을 010-1234-5678 형식으로 정했다면, 사용자가 010 1234 5678이라고 입력하더라도 데이터 파일에 저장되는 값은 010-1234-5678이다.

중복^{명확} 서로 달라야 하는 두 값이 동치인 상태. 예를 들어 데이터 파일 속의 서로 다른 두 반 레코드가 똑같이 C0001이라는 반 번호를 갖고 있다면 이는 중복이다. 아래에서 정의하는 '충돌'과 혼동하지 않기 바란다. 중복은 같아서 안 되는 값이 같아진 것이고, 충돌은 값 자체는 서로 다르지만 두 기록이 동시에 성립할 수 없는 것이다.

참조^{특정} 어떤 레코드의 필드가 다른 레코드를 가리키는 것. 예를 들어 반 레코드의 담당 강사 필드에 적힌 T0001은 강사 번호가 T0001인 강사 레코드를 참조한다. 가리키는 값이 적혀 있으나 그 대상 레코드가 실제로는 존재하지 않는 상태를 이 문서에서는 '참조가 끊어졌다'고 표현한다.

사용자 계정 (혹은 **계정**)^{특정} 프로그램에 로그인하기 위한 아이디와 비밀번호의 묶음. 모든 계정은 반드시 하나의 역할을 가지며, 한 계정이 둘 이상의 역할을 갖는 일은 없다.

역할^{특정} 계정이 사용할 수 있는 기능의 범위를 결정하는 구분. 원장, 강사, 학생 세 가지만 존재한다.

원장^{특정} 역할이 원장인 계정으로 로그인한 사용자. 학원의 모든 데이터를 등록하고 수정할 수 있는 관리자에 해당하며, 이 프로그램에서 가장 넓은 권한을 가진다. 원장 계정은 하나만 존재한다.

강사^{특정} 역할이 강사인 계정으로 로그인한 사용자. 또한 문맥에 따라, 로그인 여부와 무관하게 학원에 등록되어 반을 담당하는 사람 자체를 가리키기도 한다.

학생^{특정} 역할이 학생인 계정으로 로그인한 사용자. 또한 문맥에 따라, 로그인 여부와 무관하게 학원에 등록되어 반을 수강하는 사람 자체를 가리키기도 한다.

사용자^{명확} 학생, 강사, 원장을 모두 아울러 가리키는 말. 로그인 여부나 역할과 관계없이 이 프로그램을 사용하고 있는 사람을 통칭할 때 쓴다.

과목^{특정} 학원이 가르치는 내용의 종류. '수학', '영어' 따위가 이에 해당한다. 과목은 그 자체로 시간표에 자리를 차지하지 않으며, 누가 언제 가르치는지에 대한 정보도 갖지 않는다.

교시^{특정} 하루를 일정한 길이로 나눈, 수업 시간을 세는 단위. 각 교시가 실제로 몇 시 몇 분에 시작하고 끝나는지는 4장에서 정한다. 이 문서에서 수업 시간을 나타낼 때에는 언제나 시각이 아니라 요일과 교시로 나타낸다.

반^{명확} 하나의 과목을 한 명의 강사가 정해진 요일과 교시에 가르치기로 하여 개설한 단위. 세간에서 학급, 강좌, 클래스 등 여러 이름으로 불리는 대상을 이 문서에서는 오직 반으로만 부른다. 반은

시간표에 자리를 차지하며, 학생은 과목이 아니라 반을 수강한다. 예를 들어 '수학'은 과목이고, '월요일 3교시부터 4교시까지 박 강사가 가르치는 수학A'는 반이다.

수업^{특정} 하나의 반이 특정 날짜에 한 차례 열리는 것. 예를 들어 매주 월요일에 열리는 어떤 반이 개강일부터 종강일까지 15주 동안 지속된다면, 그 반은 하나지만 그 반의 수업은 15개다. 반과 혼동하지 않기 바란다. 1차 개발 범위에서는 수업을 데이터 파일에 저장하지 않으며, 출결과 보강을 다루게 될 2차에서 사용한다.

정원^{특정} 하나의 반에 수강 등록할 수 있는 학생 수의 상한. 1 이상의 정수다.

개강일^{특정} 어떤 반의 첫 수업이 열리는 날짜.

종강일^{특정} 어떤 반의 마지막 수업이 열리는 날짜. 같은 반의 개강일보다 앞설 수 없으며, 개강일과 같은 날일 수는 있다.

수강 등록 (혹은 등록)^{특정} 어떤 학생이 어떤 반을 수강하기로 데이터 파일에 기록된 상태. 학생이 반의 정보를 조회하기만 한 것은 수강 등록이 아니며, 등록을 신청했으나 정원 초과 등의 이유로 거부된 것도 수강 등록이 아니다.

수강 취소^{특정} 이미 이루어진 수강 등록을 더 이상 유효하지 않은 상태로 바꾸는 것. 취소된 기록은 데이터 파일에서 지워지지 않고 취소된 상태로 남는다. 따라서 취소된 등록은 그 반의 등록 인원을 셀 때 포함하지 않는다.

충돌^{특정} 한 사람이 같은 요일의 같은 교시에 둘 이상의 반에 관여하게 되는 상태. 한 강사가 같은 요일의 같은 교시에 두 반을 담당하는 경우와, 한 학생이 같은 요일의 같은 교시에 두 반을 수강하는 경우가 이에 해당한다. 반면 서로 다른 강사가 담당하는 두 반이 같은 요일의 같은 교시에 열리는 것은 충돌이 아니다.

가상 현재 일시 (혹은 **현재 일시**)^{신규} 이 프로그램이 '지금'으로 간주하는 날짜와 시각. 실제 컴퓨터 시각과는 무관하게 데이터 파일에 저장되어 있으며, 사용자가 직접 지정한다. 수강 신청이 가능한 기간인지, 어떤 반이 이미 개강했는지 등 시간에 의존하는 모든 판단은 실제 시각이 아니라 이 값을 기준으로 이루어진다.

전진^{변경} 가상 현재 일시를 지금 저장되어 있는 값보다 더 나중의 일시로 바꾸는 것. 이 프로그램에서 가상 현재 일시는 전진만 가능하고 더 이른 일시로는 바꿀 수 없으며, 같은 일시로 다시 지정하는 것도 전진에 포함한다.

2. 기본 사항

2.1 작동 환경

이 프로그램은 Python 3.10 이상의 인터프리터로 실행되며, 그 외의 환경에서는 동작을 보장하지 않는다. Python 표준 라이브러리 외의 외부 라이브러리는 사용하지 않는다.

Windows의 명령 프롬프트와 PowerShell, Linux와 macOS의 터미널에서 모두 동작한다. 화면 출력은 고정폭 글꼴 사용을 전제로 하며, 터미널 환경에 따라 한글과 영문의 표시 폭 차이로 표의 열 정렬이 달라질 수 있다. 각 필드의 값은 길이에 관계없이 잘라내지 않고 전체를 출력하며, 한 행이 터미널 가로 폭을 초과하는 경우 터미널의 자동 줄바꿈을 따른다. 출력 예시의 공백과 열 정렬은 형식 이해를 위한 예시이며, 정확한 공백 수 자체를 출력 규칙으로 보지 않는다. 한 시점에 한 명의 사용자만이 프로그램을 사용하는 것을 전제한다. 같은 데이터 파일을 두 개 이상의 프로그램이 동시에 열어 사용하는 경우 정상 동작을 보장하지 않는다.

2.2 프로그램 구성

프로그램은 주 실행 파일 하나와 데이터 파일 여러 개로 구성된다. 데이터 파일은 주 실행 파일과 같은 디렉토리 아래의 data 디렉토리에 둔다.

main.py	주 실행 파일
data/datetime.txt	현재 가상 현재 일시
data/users.txt	사용자 계정
data/students.txt	학생
data/teachers.txt	강사
data/subjects.txt	과목
data/classes.txt	반
data/enrollments.txt	수강 등록

모든 데이터 파일은 BOM 없는 UTF-8 인코딩으로 저장한다. 사용자가 데이터 파일을 직접 편집할 때에도 이 인코딩을 지켜야 한다.

프로그램이 실행되고 있는 도중에 데이터 파일을 편집하는 경우 정상 동작을 보장하지 않는다.

2.3 프로그램 설치 및 실행

주 실행 파일을 임의의 비어 있는 디렉토리에 복사하는 것으로 설치가 끝난다. data 디렉토리와 그 안의 데이터 파일들은 프로그램이 처음 실행될 때 자동으로 생성된다.

해당 디렉토리에서 다음과 같이 실행한다.

python [main.py](#)

데이터 파일을 생성하거나 읽고 쓸 권한이 없는 경우, 프로그램은 오류 메시지를 출력한 뒤 즉시 종료한다. 자세한 절차는 5.4절에서 다룬다.

2.3.1 최초 실행

프로그램을 처음 실행하여 `data` 디렉토리가 존재하지 않는 경우 프로그램은 다음 순서로 초기 데이터를 생성한다.

1. `data` 디렉토리를 생성한다.
2. 각 데이터 파일과 필요한 헤더를 생성한다.
3. 원장 계정의 사용자 ID와 비밀번호를 입력받아 `users.txt`에 저장한다.
4. 최초 가상 현재 일시를 입력받아 `datetime.txt`에 저장한다.
5. 생성된 파일에 대하여 기본 무결성 검사를 수행한다.
6. 이상이 없으면 로그인 화면을 출력한다.

최초 실행 시에는 기존 가상 현재 일시가 없으므로 가상 현재 일시 입력을 생략할 수 없다.

최초 생성 이후의 실행에서는 저장된 `datetime.txt` 값을 그대로 사용하며, 프로그램 시작 시마다 새로운 가상 현재 일시를 입력받지 않는다.

최초 실행 절차는 3단계와 4단계가 모두 끝나 `users.txt`에 원장 계정이 기록되고 `datetime.txt`가 생성된 시점에 완료된 것으로 본다. 다음 실행에서 `data` 디렉토리는 존재하지만 `users.txt`에 원장 계정이 없거나 `datetime.txt`가 없는 경우, 최초 실행이 완료되지 않은 것으로 보고 생성된 파일을 모두 삭제한 뒤 본 절차를 처음부터 다시 수행한다.

2.4 사용자 역할과 권한

사용자는 로그인한 뒤 계정에 기록된 역할에 따라 사용할 수 있는 기능이 달라진다.

로그인하기 전에는 로그인 외의 기능을 사용할 수 없으며, 학원 데이터는 조회조차 할 수 없다. 각 역할이 사용할 수 있는 기능은 다음과 같다.

기능	학생	강사	원장
가상 현재 일시 변경	×	×	○
개설된 반 조회	○	○	○
수강 신청·취소	○	×	×
본인 수강 목록 조회	○	×	×
담당 반 수강생 조회	×	○	○
학생·강사·과목 등록 및 수정	×	×	○
반 개설·수정·폐강	×	×	○
전체 수강 등록 조회·취소	×	×	○
무결성 검사 실행	×	×	○
계정 비밀번호 관리	×	×	○

강사는 자신에게 배정된 반의 수강생만 조회할 수 있고, 원장은 모든 반의 수강생을 조회할 수 있다.

계정은 모두 원장이 생성한다. 학생이나 강사를 등록하면 그 사람의 사용자 계정이 함께 생성된다. 단, 원장 계정은 프로그램이 최초로 실행될 때 한 번만 생성한다.

학생과 강사는 가상 현재 일시를 조회할 수는 있으나 직접 변경할 수 없다. 가상 현재 일시 변경 권한은 원장에게만 부여한다. 단, 프로그램이 최초로 실행될 때에는 아직 계정이 없으므로, 2.3.1절에 따라 로그인 이전에 최초 가상 현재 일시를 한 번 입력받는다.

2.5 프로젝트 범위

본 프로그램의 1차 구현 범위는 학원 관리에 필요한 기본적인 사용자 및 수강 관리 기능으로 한정한다.

포함 범위

- 학생·강사·원장 사용자 관리
- 로그인 및 역할별 메뉴 제공
- 과목 및 반 관리
- 반의 수업 일정 및 정원 관리
- 학생의 수강 신청 및 수강 취소
- 학생·강사의 수강 관련 정보 조회
- 원장의 전체 수강 등록 내역 조회 및 관리
- 데이터 파일 저장 및 무결성 검사

제외 범위

다음 기능은 1차 구현 범위에 포함하지 않는다.

- 출결 관리
- 보강 관리
- 수강료 관리
- 결제 및 환불 관리
- 강의실 관리
- 문자·알림 발송
- 성적 및 평가 관리

위 제외 기능은 향후 확장 가능한 기능으로 고려할 수 있으나, 본 1차 구현에서는 데이터 구조와 기능을 정의하거나 구현하지 않는다.

3. 프로그램 사용 흐름도

프로그램 사용 흐름, 즉, 프로그램 사용자 관점에서 프로그램을 사용하는 단계별 경로는 17,18 쪽의 그림 1~4와 같다.

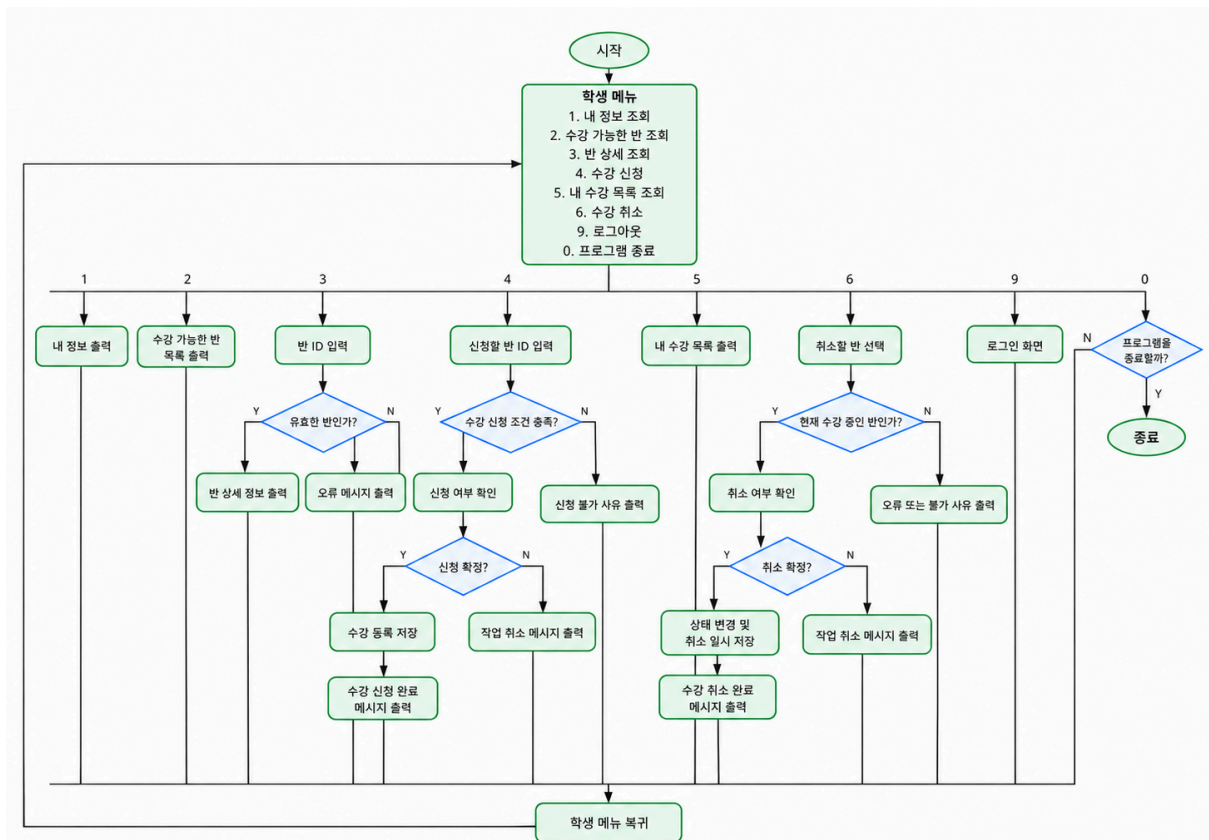
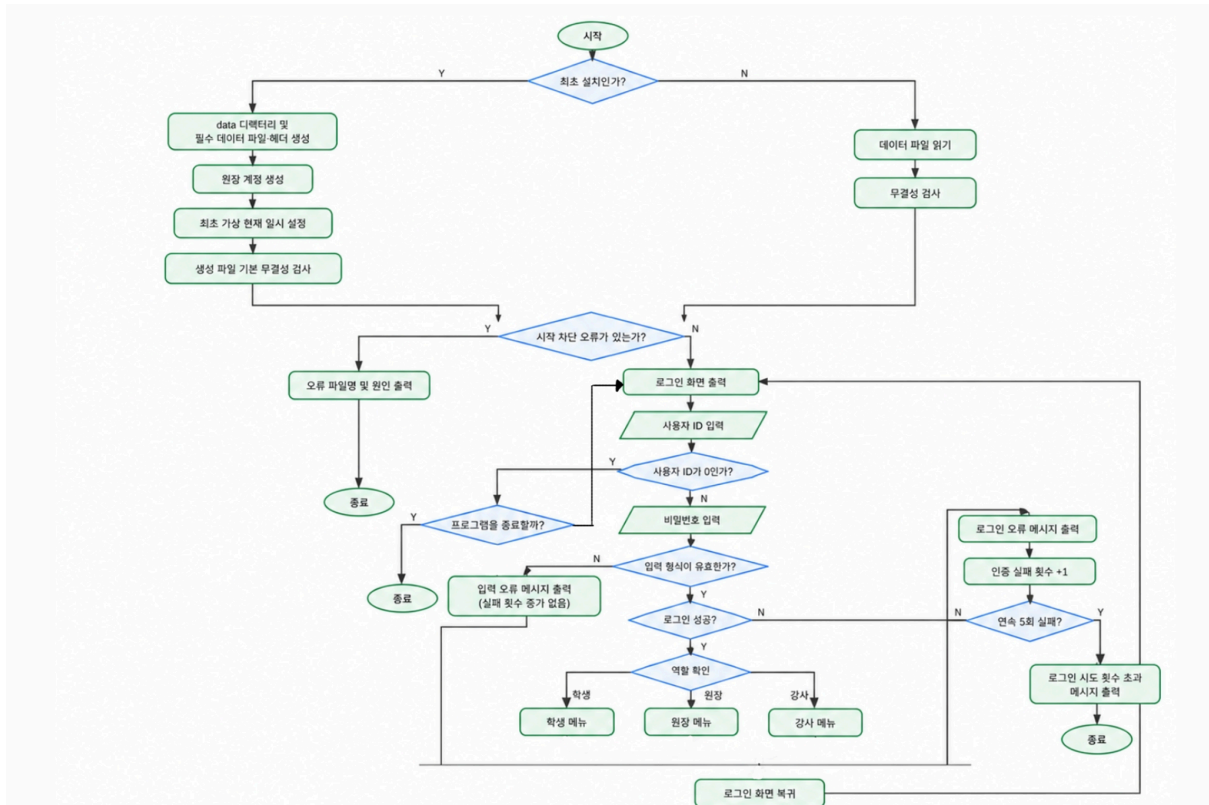


그림 2: 학생 사용 흐름도

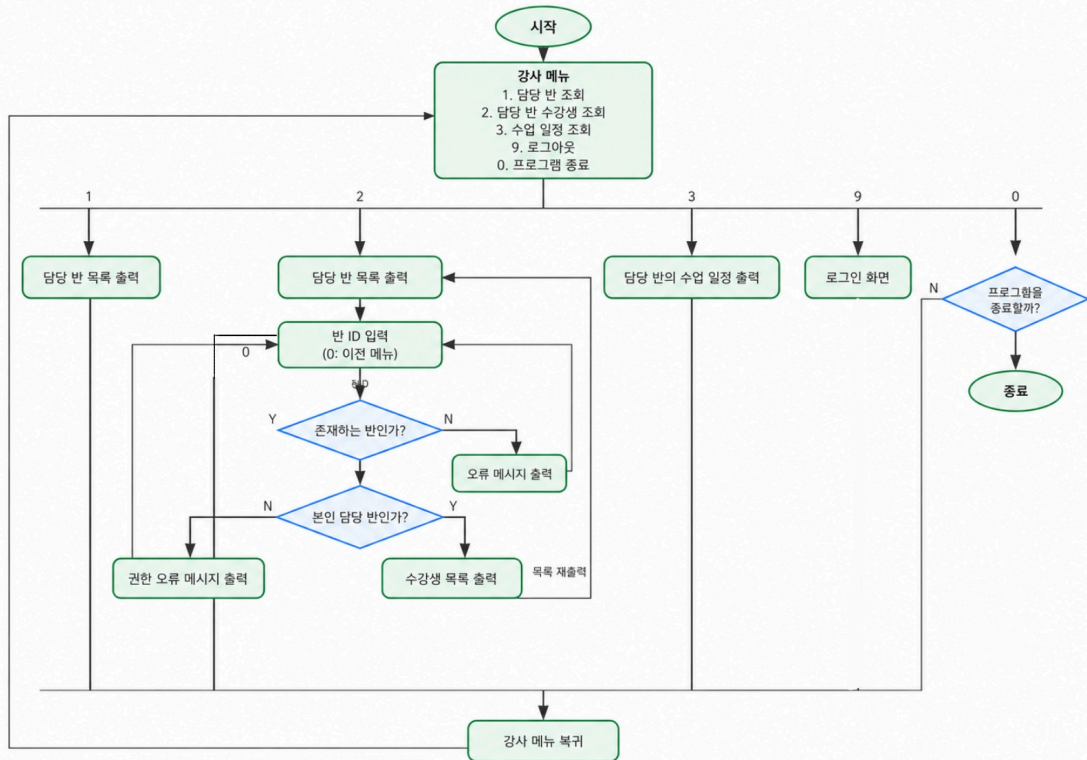


그림 3: 강사 사용 흐름도

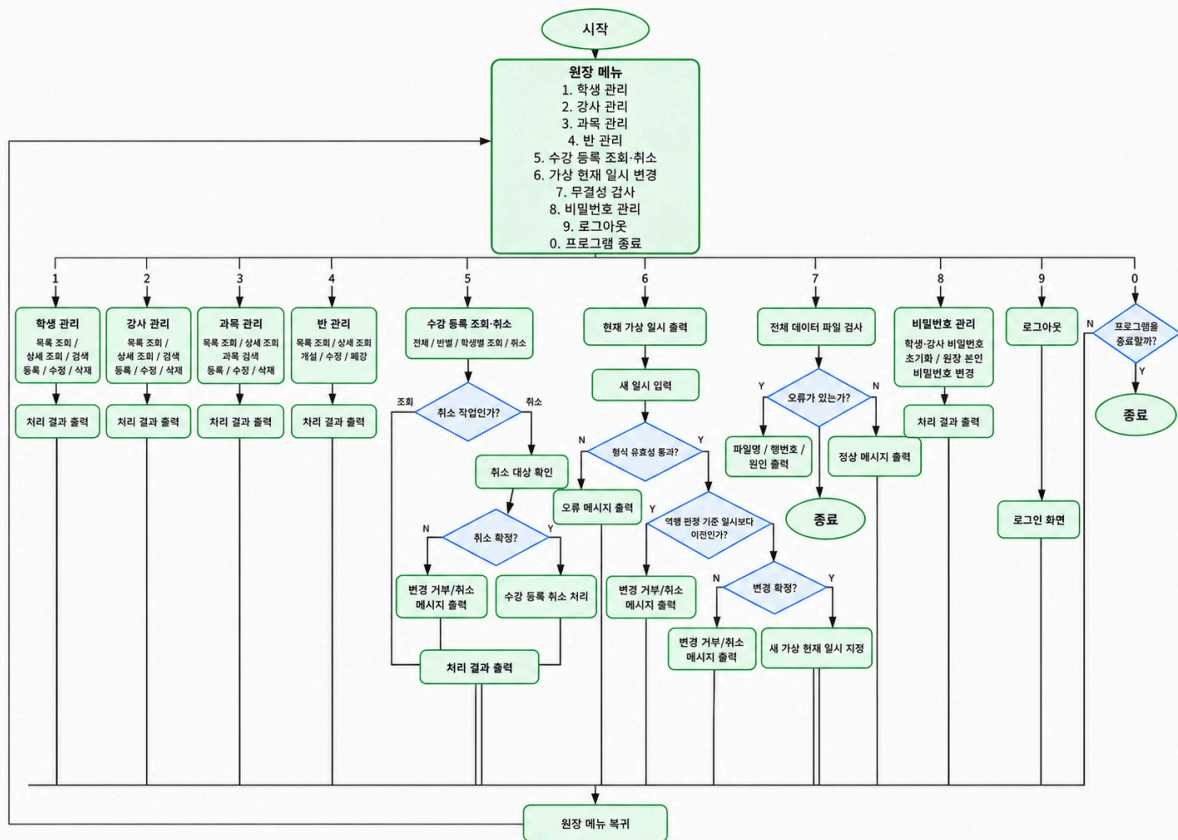


그림 4: 원장 사용 흐름도

3.1 학생

학생은 자신의 계정으로 로그인한 후 학생 메뉴를 통해 수업 조회, 수강 신청, 수강 내역 확인 및 수강 취소 기능을 이용한다.

전체 흐름 : 학생 로그인 → 학생 메뉴 → 수업 조회 → 수업 선택 → 수강 신청 조건 확인 → 수강 신청 완료 → 내 수강 목록 확인 → 필요 시 수강 취소 → 로그아웃

프로그램 시작 시 data 디렉토리의 존재 여부로 최초 설치인지 확인한다. 최초 설치인 경우 원장 계정과 최초 가상 현재 일시를 설정한다. 기존 데이터가 있는 경우 저장된 데이터를 읽어 무결성을 검사한다. 시작을 차단하는 오류가 발견되면 오류가 발생한 파일과 원인을 출력하고 종료한다. 검사를 통과하면 로그인 화면을 출력한다. 로그인 성공 후에는 계정의 역할에 따라 해당 메뉴로 이동한다.

전체 흐름은 대표적인 사용 예시이며, 기능을 반드시 나열된 순서대로 실행해야 하는 것은 아니다. 사용자는 역할별 메뉴에서 원하는 기능을 선택한다. 조회 또는 변경 작업이 끝나거나 작업을 취소하면 해당 역할의 메뉴로 돌아간다. 입력 오류는 해당 입력 단계에서 다시 입력받고, 저장 실패는 실패 메시지를 출력한 뒤 해당 메뉴로 돌아간다.

3.1.1 로그인

- 아이디와 비밀번호를 입력하여 로그인한다.
- 로그인한 사용자의 역할이 학생인지 확인한다.
- 로그인에 성공하면 학생 메뉴를 출력한다.

3.1.2 학생 메뉴 선택

학생은 다음 기능을 선택할 수 있다.

- 내 정보 조회
- 수강 가능한 반 조회
- 반 상세 조회

- 수강 신청
- 내 수강 목록 조회
- 수강 취소
- 로그아웃

3.1.3 내 정보 조회

로그인한 학생 본인의 학생 ID와 이름을 조회한다.

자세한 내용은 6.4.1의 내용을 참고한다.

3.1.4 수강 가능한 반 조회

현재 수강 신청이 가능한 반 목록을 조회한다.

반 ID, 반 이름, 과목, 담당 강사, 수업 요일, 교시, 현재 수강 인원, 정원 등의 정보를 확인한다.

원하는 반을 선택하여 상세 정보를 확인할 수 있다.

3.1.5 반 상세 조회

수강 가능한 반 목록에서 반 ID를 입력하여 해당 반의 상세 정보를 조회한다. 반 ID, 반 이름, 과목명, 담당 강사, 수업 요일, 교시, 수업 기간, 현재 수강 인원, 정원 등을 확인할 수 있다.

3.1.6 수강 신청

학생은 신청할 반 ID를 입력한다. 프로그램은 반의 존재 여부, 학생의 재원 여부, 반의 개설 상태, 신청 가능 기간, 중복 수강 여부, 정원 및 다른 수강 반과의 시간 충돌 여부를 확인한다. 조건을 만족하지 않으면 신청을 거부하고 사유를 출력한다. 조건을 만족하면 반 정보를 표시하고 신청 여부를 확인받는다. 학생이 동의하면 새로운 수강 등록을 생성하고 등록 일시에 가상 현재 일시를 기록한다.

저장에 성공하면 완료 메시지를 출력하고 학생 메뉴로 돌아간다. 확인을 취소하면 데이터를 변경하지 않는다. 저장 실패 시에는 실패 메시지를 출력한다.

3.1.7 내 수강 목록 조회

현재 학생이 신청한 수업 목록을 조회한다.

반 ID, 반 이름, 과목, 담당 강사, 수업 요일 및 교시, 수강 상태 등의 정보를 확인할 수 있다.

3.1.8 수강 취소

자신의 수강 신청 목록에서 취소할 수업을 선택한다.

해당 수업의 수강 취소 가능 여부를 확인한다.

취소가 가능하면 해당 수강 등록의 상태를 "**취소**"로 변경하고, 취소 일시에 현재 가상 현재 일시를 기록한다.

입력한 반에 대해 로그인한 학생 본인의 수강중 등록이 존재하는지 확인한다. 대상이 없으면 오류를 출력하고 다시 입력받는다. 대상이 있으면 반 정보를 출력하고 취소 여부를 확인받는다. 취소에 동의하면 등록 상태를 취소로 변경하고 취소 일시에 가상 현재 일시를 기록한다. 저장 성공 후 완료 메시지를 출력하며, 취소된 등록 기록은 삭제하지 않는다.

3.1.9 로그아웃

학생 메뉴에서 로그아웃을 선택하면 현재 학생 계정을 로그아웃한다.

로그아웃 후 6.2.1절의 로그인 화면으로 돌아간다.

3.2 강사

강사는 자신의 계정으로 로그인한 후 강사 메뉴를 통해 자신이 담당하는 반과 시간표, 수강생 정보를 조회하는 기능을 이용한다.

전체 흐름: 강사 로그인 → 강사 메뉴 → 담당 반 조회 → 담당 반 수강생 조회 → 수업 일정 조회 → 로그아웃

3.2.1 로그인

아이디와 비밀번호를 입력하여 로그인한다.

로그인한 사용자의 역할이 강사인지를 확인한다.

로그인에 성공하면 강사 메뉴를 출력한다.

3.2.2 강사 메뉴 선택

강사는 다음 기능을 선택할 수 있다.

- 담당 반 조회
- 담당 반 수강생 조회
- 수업 일정 조회
- 로그아웃

3.2.3 담당 반 조회

로그인한 강사가 담당하고 있는 반 목록을 조회한다. 반 ID, 반 이름, 과목명, 담당 강사, 수업 요일, 교시, 수업 기간, 반 상태, 현재 수강 인원, 정원 등의 정보를 확인할 수 있다. 자신에게 담당 강사로 배정된 반만 조회 대상에 포함된다.

3.2.4 수업 일정 조회

로그인한 강사가 담당하고 있는 반의 수업 일정을 요일과 교시 순으로 정렬하여 조회한다. 자신이 담당하지 않는 반의 일정은 조회 대상에 포함하지 않는다.

3.2.5 담당 반 수강생 조회

자신이 담당하는 반을 선택하여 현재 수강 중인 학생 목록을 조회한다. 학생 ID, 학생 이름 등의 정보를 확인할 수 있다. 수강 취소된 학생은 현재 수강생 목록에서 제외한다. 자신이 담당하지 않는 반의 수강생은 조회할 수 없다.

강사는 조회할 반 ID를 입력한다. 프로그램은 해당 반의 존재 여부와 로그인한 강사가 해당 반의 담당 강사인지를 확인한다. 존재하지 않는 반이면 오류를 출력하고 다시 입력받는다. 본인이 담당하지 않는 반이면 권한 오류를 출력하며 수강생 정보를 표시하지 않는다. 본인의 담당 반이면 수강중 상태인 등록의 학생 목록을 출력한다. 해당 학생이 없으면 수강생이 없다는 메시지를 출력한다.

3.2.6 로그아웃

강사 메뉴에서 로그아웃을 선택하면 현재 강사 계정을 로그아웃한다. 로그아웃 후 프로그램의 로그인 화면으로 돌아간다.

3.3 원장

원장은 자신의 계정으로 로그인한 후 원장 메뉴를 통해 학생·강사·과목·반 데이터의 등록·수정·삭제, 전체 수강 등록 조회 및 취소, 가상 현재 일시 변경, 무결성 검사를 수행한다.

전체 흐름: 원장 로그인 → 원장 메뉴 → 학생·강사·과목·반 관리 → 수강 등록 조회·취소 → 가상 현재 일시 변경 → 무결성 검사 → 로그아웃

3.3.1 로그인

아이디와 비밀번호를 입력하여 로그인한다.
로그인한 사용자의 역할이 원장인지 확인한다.
로그인에 성공하면 원장 메뉴를 출력한다.

3.3.2 원장 메뉴 선택

원장은 다음 기능을 선택할 수 있다.

- 학생 관리
- 강사 관리
- 과목 관리
- 반 관리
- 수강 등록 조회·취소
- 가상 현재 일시 변경
- 무결성 검사
- 비밀번호 관리
- 로그아웃

등록 작업은 필드 입력, 형식·의미 검사, 최종 확인, 저장 순서로 진행한다. 수정·삭제·폐강 작업은 대상 ID 입력, 대상 존재 확인, 변경 가능 조건 검사, 최종 확인, 저장 순서로 진행한다. 확인을 취소하면 데이터를 변경하지 않는다. 관련 파일을 여러 개 변경하는 작업은 전체 변경을 하나의 작업으로 처리하며, 일부 파일만 변경된 상태가 남지 않도록 한다. 저장이 완료된 경우에만 성공 메시지를 출력한다.

3.3.3 학생 관리

등록된 학생 목록과 상세 정보를 조회하고, 새로운 학생을 등록하거나 기존 학생의 정보를 수정, 삭제한다. 학생의 이름·연락처·학생 상태(재원·퇴원)를 관리한다.

학생을 등록하면 그 학생의 사용자 계정이 함께 생성된다. 학생 상태를 퇴원으로 변경하면 그 학생의 "수강중" 상태의 수강 등록이 모두 자동으로 취소 처리된다. 수강 등록 이력이 하나라도 있는 학생은 삭제할 수 없으며, 퇴원 상태로 유지한다.

3.3.4 강사 관리

등록된 강사 목록과 상세 정보를 조회하고, 새로운 강사를 등록하거나 기존 강사의 정보를 수정, 삭제한다. 강사의 이름·연락처·강사 상태(재직·퇴직)를 관리하며, 담당 반이 있는 강사는 삭제할 수 없고 강사 상태를 퇴직으로 변경하는 방식으로 처리한다.

3.3.5 과목 관리

등록된 과목 목록과 상세 정보를 조회하고, 새로운 과목을 등록하거나 기존 과목의 이름을 수정, 삭제한다. 그 과목으로 개설된 반이 하나라도 있으면 삭제할 수 없다.

3.3.6 반 관리

등록된 반 목록과 상세 정보를 조회하고, 새로운 반을 개설하거나 기존 반의 정보를 수정, 폐강한다. 반을 개설하거나 수정할 때에는 담당 강사의 시간 충돌 여부, 반 이름 중복, 정원과 현재 수강 인원의 관계 등을 검사한다. 반을 폐강하면 그 반에 연결된 "수강중" 상태의 수강 등록도 함께 취소 처리된다.

3.3.7 수강 등록 조회·취소

원장은 전체 수강 등록 내역을 조회하고 필요한 경우 수강 등록을 취소할 수 있다.

3.3.8 가상 현재 일시 변경

원장은 프로그램의 가상 현재 일시를 새로운 일시로 변경할 수 있다. 상세 흐름은 6.3절에서 정의한다. 가상 현재 일시가 변경되면 이후 수강 신청 가능 여부·개강 여부 등 시간에 의존하는 판단이 새 값을 기준으로 이루어진다.

현재 가상 일시를 표시하고 새 일시를 입력받는다. 새 일시의 형식과 유효성을 검사하고, 현재 가상 일시 및 역행 검사 기준이 되는 기록 일시보다 이전이면 변경을 거부한다. 같은 일시는 허용한다. 검사를 통과하면 변경 여부를 확인받고 저장한다. 저장 성공 후 이후의 시간 관련 판단에 새 값을 적용한다. 확인을 취소하면 기존 값을 유지한다.

3.3.9 무결성 검사

원장은 데이터 파일 전체의 문법·참조·의미 규칙을 다시 검사할 수 있다.

검사 결과 이상이 없으면 정상 메시지를 출력한다. 오류가 있으면 파일명, 가능한 경우 행 번호, 오류 코드와 원인을 출력한다. 프로그램은 오류 데이터를 자동으로 수정하지 않는다. 관리 메뉴에서 수정 가능한 항목과 외부 텍스트 편집기로 정정해야 하는 항목을 구분하여 안내한다. 외부 편집이 필요한 경우 프로그램을 종료한 뒤 수정하고 다시 실행하도록 안내한다.

이외의 상세 흐름과 검사 항목은 5.4절에서 정의한다.

3.3.10 로그아웃

원장 메뉴에서 로그아웃을 선택하면 현재 원장 계정을 로그아웃한다. 로그아웃 후 프로그램의 로그인 화면으로 돌아간다.

4. 데이터 요소

이 프로그램에서 사용하는 주요 데이터 요소에는 식별자, 이름, 비밀번호, 사용자 역할, 연락처, 상태, 날짜, 시각, 일시, 요일, 수업 일정, 정원이 있다. 각 데이터 요소에 대해서는 해당 요소의 의미와 용도를 설명하고, 문자열로 표현되는 요소의 경우 문법적으로 올바른 형식을 명시한다. 문법적으로 올바른 값이 추가로 만족해야 하는 조건이 있는 경우 이를 의미 규칙으로 구분하여 명시한다.

두 값이 같은 값을 나타내는지 판단할 필요가 있는 요소에는 동치 비교 규칙을 명시한다. 사용자가 해당 요소를 기준으로 데이터를 검색하는 기능이 있는 경우에는 검색어와 저장된 값이 합치되는 조건을 검색 규칙으로 명시한다. 하나의 값을 여러 형태로 표현할 수 있는 경우에는 필요에 따라 해석 방법이나 표준형을 함께 정의한다.

4.1 식별자

식별자는 프로그램에서 관리하는 사용자 계정, 학생, 강사, 과목, 반, 수강 등록을 서로 구별하고 다른 데이터에서 참조하기 위해 사용하는 문자열이다.

식별자는 사용자 ID와 내부 식별자로 구분한다. 사용자 ID는 사용자가 프로그램에 로그인할 때 직접 입력하는 식별자이고, 내부 식별자는 학생, 강사, 과목, 반, 수강 등록 등의 데이터를 식별하거나 다른 데이터에서 해당 데이터를 참조하기 위해 사용하는 식별자이다. 식별자의 종류가 서로 다른 경우에는 문자열의 일부가 같더라도 같은 식별자로 간주하지 않는다.

4.1.1 사용자 ID

사용자 ID는 원장, 강사, 학생이 프로그램에 로그인할 때 자신의 사용자 계정을 식별하기 위해 사용하는 문자열이다. 하나의 사용자 계정은 하나의 사용자 ID를 가지며, 서로 다른 사용자 계정이 같은 사용자 ID를 가질 수 없다.

문법 형식: 문법적으로 올바른 사용자 ID는 다음 조건을 모두 만족하는 문자열이다.

- 길이는 4 이상 16 이하
- 첫 문자는 로마자 소문자 "a"부터 "z" 중 하나
- 두 번째 문자부터 마지막 문자까지는 로마자 소문자 "a"부터 "z" 또는 숫자 "0"부터 "9" 중 하나
- 표준공백, 탭, 개행 및 그 밖의 특수문자를 모두 포함하지 않음

예를 들어 "minsu", "std01", "teacher2", "a1b2c3d"는 모두 올바른 사용자 ID이지만, "01minsu", "KimMinSU", "min su", "abc", "abcd!"는 올바른 사용자 ID가 아니다.

"01minsu"는 첫 문자가 숫자이고, "KimMinSU"는 로마자 대문자를 포함한다. "min su"는 공백을 포함하고, "abc"는 길이가 4 미만이다. "abcd!"는 허용되지 않는 "!" 문자를 포함한다.

의미 규칙: 문법적으로 올바른 사용자 ID는 현재 등록되어 있는 다른 사용자 계정의 사용자 ID와 중복되지 않아야 한다.

예를 들어 "std01"이라는 사용자 ID를 가진 계정이 이미 존재하는 경우, 새로운 계정의 사용자 ID로 "std01"을 다시 사용할 수 없다.

동치 비교: 두 사용자 ID의 문자열 전체가 완전히 일치하는 경우에만 서로 동치인 사용자 ID로 간주한다.

예를 들어 "student01"과 "student01"는 서로 동치이다. 반면 "student01"과 "student02"는 서로 동치가 아니다.

검색: 사용자 ID 자체를 검색어로 사용하여 사용자 계정을 부분 검색하는 기능은 제공하지 않는다. 따라서 별도의 검색어 합치 규칙은 두지 않는다.

4.1.2 내부 식별자

내부 식별자는 학생, 강사, 과목, 반, 수강 등록 등의 데이터를 서로 구별하거나 다른 데이터에서 해당 데이터를 참조하기 위해 사용하는 문자열이다.

내부 식별자의 종류에는 학생 ID, 강사 ID, 과목 ID, 반 ID, 수강 등록 ID가 있다.

종류	대상	형식 예
학생 ID	학생	“S0001”
강사 ID	강사	“T0001”
과목 ID	과목	“P0001”
반 ID	반	“C0001”
수강 등록 ID	수강 등록	“E0001”

반 데이터에서 담당 강사를 나타낼 때에는 강사 ID를 사용하고, 수강 등록 데이터에서 수강 학생과 대상 반을 나타낼 때에는 각각 학생 ID와 반 ID를 사용한다.

문법 형식: 문법적으로 올바른 내부 식별자는 다음 형식 중 하나에 부합하는 문자열이다.

S 〈숫자 4개〉

T 〈숫자 4개〉

P 〈숫자 4개〉

C 〈숫자 4개〉

E 〈숫자 4개〉

각 접두사는 다음 종류의 데이터를 나타낸다.

S: 학생

T: 강사

P: 과목

C: 반

E : 수강 등록

숫자 부분은 정확히 네 자리이며 선행 0을 허용한다.

단, 숫자 부분이 0000인 식별자는 발급 전 상태를 나타내는 값으로 예약되어 있으므로 올바른 내부 식별자로 간주하지 않는다.

다음은 문법적으로 올바른 내부 식별자의 예이다.

예를 들어 “S0001”, “T0027”, “P0100”, “C0012”, “E9999”는 모두 올바른 내부 식별자이지만, “S1”, “s0001”, “S00001”, “S 0001”, “A0001”은 올바른 내부 식별자가 아니다.

“S1”은 숫자 부분이 네 자리가 아니고, “s0001”은 지정된 대문자 접두사를 사용하지 않는다.

“S00001”은 숫자 부분이 다섯 자리이고, “S 0001”은 공백을 포함한다. “A0001”은 정의되지 않은 접두사를 사용한다.

의미 규칙: 내부 식별자는 다음 조건을 만족해야 한다.

- 같은 종류의 데이터에서는 서로 중복될 수 없다.
- 다른 데이터에서 내부 식별자를 참조하는 경우, 해당 식별자를 가진 데이터가 실제로 존재해야 한다.

예를 들어 학생 데이터에 이미 “S0001”이 있다면 다른 학생에게 다시 “S0001”을 부여할 수 없다. 또, 반 데이터의 담당 강사 ID가 “T0003”이라면 강사 데이터에 “T0003”이라는 강사 ID를 가진 강사가 존재해야 한다. 수강 등록 데이터의 학생 ID가 “S0005”이고 반 ID가 “C0002”라면 각각의 ID에 해당하는 학생과 반이 모두 존재해야 한다. 존재하지 않는 내부 식별자를 참조하는 경우에는 데이터 무결성 오류로 처리한다.

동치 비교: 두 내부 식별자의 문자열 전체가 완전히 일치하는 경우에만 서로 동치인 식별자로 간주한다.

예를 들어 “S0001”과 “S0001”은 서로 동치이다. 반면 “S0001”과 “S0010”은 서로 동치가 아니다.

내부 식별자의 숫자 부분을 별도의 정수로 해석하여 동치를 비교하지 않는다. 따라서 “S0001”과 “S1”을 같은 학생 ID로 간주하지 않으며, “S1”은 문법적으로 올바른 학생 ID 자체가 아니다.

검색: 내부 식별자를 이용하여 특정 학생, 강사, 과목, 반 또는 수강 등록을 지정할 때에는 입력한 식별자와 저장된 식별자가 동치인 경우에만 합치된 것으로 간주한다. 부분문자열 합치는 허용하지 않는다.

예를 들어 저장된 반 ID가 “C0012”인 경우, 입력값이 “C0012”일 때에만 합치(지정 성공)되고, “0012”, “C12”, “C001” 등의 입력값은 모두 불합치 처리된다.

4.2 이름

이름은 학생과 강사의 성명, 과목의 명칭, 반의 명칭을 나타내는 문자열이다. 이름의 사용 대상에 따라 사람 이름, 과목명, 반 이름으로 구분한다.

4.2.1 사람 이름

사람 이름은 학생 또는 강사의 성명을 나타내는 문자열이다. 서로 다른 학생이나 강사가 같은 이름을 가질 수 있으므로 사람 이름은 개인을 고유하게 식별하기 위한 값으로 사용하지 않는다. 학생과 강사의 고유한 식별에는 각각 학생 ID와 강사 ID를 사용한다.

문법 형식: 문법적으로 올바른 사람 이름은 다음 조건을 모두 만족하는 문자열이다.

- 길이는 1 이상 20 이하이다.
- 한글 완성형 문자, 로마자 대문자 "A"부터 "Z", 로마자 소문자 "a"부터 "z", 표준공백만 사용할 수 있다.
- 첫 문자와 마지막 문자는 표준공백일 수 없다.
- 표준공백이 두 개 이상 연속해서 등장할 수 없다.
- 탭과 개행은 포함할 수 없다.

예를 들어 “김민수”, “김 민수”, “Min Su Kim”, “Alice”는 모두 올바른 사람 이름이지만, “ 김민수”, “김민수 ”, “김 민수”, “김민수123”은 올바른 사람 이름이 아니다.

“김민수”와 “김민수 ”는 각각 선행·후행 공백이 존재한다. “김 민수”는 두 개의 표준공백이 연속해서 등장하며, “김민수123”은 허용되지 않는 숫자를 포함한다.

의미 규칙: 사람 이름에는 추가적인 의미 규칙을 두지 않는다. 서로 다른 학생 또는 강사가 같은 이름을 갖는 것을 허용한다.

동치 비교: 두 사람 이름은 문자열 전체가 완전히 일치하는 경우에만 서로 동치인 이름으로 간주한다. 따라서 공백의 유무와 로마자 대소문자의 차이도 동치 비교에서 구별한다.

예를 들어 “김민수”와 “김 민수”는 서로 동치가 아니며, “Alice”와 “alice”도 서로 동치가 아니다.

검색: 학생 또는 강사를 이름으로 검색할 때에는 다음 규칙에 따라 검색어와 사람 이름의 합치를 판단한다.

- 검색어와 저장된 이름에서 모든 표준공백을 무시한다.
- 로마자 대소문자를 구분하지 않는다.
- 위 두 처리를 거친 검색어가 저장된 이름의 부분문자열이면 합치된 것으로 간주한다.

예를 들어 검색어가 “김민”일 경우 “김민수”, “김 민수”는 모두 합치된다. 검색어가 “alice”일 경우 “Alice”, “ALICE”도 모두 합치된다.

4.2.2 과목명

과목명은 학원에서 운영하는 과목의 명칭을 나타내는 문자열이다. 과목 자체의 고유한 식별에는 과목명이 아닌 과목 ID를 사용한다.

문법 형식: 문법적으로 올바른 과목명은 다음 조건을 모두 만족하는 문자열이다.

- 길이는 1 이상 20 이하이다.
- 숫자, 한글 완성형 문자, 로마자 대문자 "A"부터 "Z", 로마자 소문자 "a"부터 "z", 표준공백만 사용할 수 있다.
- 첫 문자와 마지막 문자는 표준공백일 수 없다.
- 표준공백이 두 개 이상 연속해서 등장할 수 없다.

- 탭과 개행은 포함할 수 없다.

예를 들어 “수학”, “중등 수학”, “영어”, “Math 1”, “고등수학2”는 모두 올바른 과목명이지만, “ 수학”, “중등수학 ”, “중등 수학”, “수학#1”은 올바른 과목명이 아니다.

의미 규칙: 과목명에는 추가적인 의미 규칙을 두지 않는다.

동치 비교: 두 과목명의 문자열 전체가 완전히 일치하는 경우에만 서로 동치인 과목명으로 간주한다.

검색: 과목명을 검색할 때에는 사람 이름 검색과 마찬가지로 표준공백과 로마자 대소문자를 무시하고, 검색어가 과목명의 부분문자열인 경우 합치된 것으로 간주한다.

예를 들어 검색어가 “중등수학”인 경우 “중등 수학”에 합치된다.

4.2.3 반 이름

반 이름은 하나의 개설된 반을 사용자가 쉽게 구별하기 위해 붙이는 명칭이다. 반 자체를 고유하게 식별할 때에는 반 이름이 아닌 반 ID를 사용한다.

문법 형식: 문법적으로 올바른 반 이름은 다음 조건을 모두 만족하는 문자열이다.

- 길이는 정확히 2글자이다.
- 첫 번째 문자는 로마자 대문자 "A"부터 "Z" 중 하나이다.
- 두 번째 문자는 한글 완성형 문자 "반"이다.
- 표준공백, 숫자, 로마자 소문자 및 그 밖의 문자는 사용할 수 없다.

따라서 "A반", "B반", "C반", ..., "Z반"만 문법적으로 올바른 반 이름이다.

예를 들어 "A반", "B반", "Z반"은 올바른 반 이름이지만, "a반", "AA반", "1반", "A 반", "수학A반", "심화반"은 올바른 반 이름이 아니다.

의미 규칙: 동일한 과목에 속하면서 동시에 "활성 상태"인 서로 다른 두 반은 서로 동치인 반 이름을 가질 수 없다. 서로 다른 과목에 속하는 반끼리는 같은 반 이름을 가질 수 있다.

본 규칙에서 "활성 상태"란 아직 폐강되지 않은 반을 의미한다. 같은 과목에서 이미 폐강된 반의 이름은 새로운 반에 다시 사용할 수 있다. 폐강된 반과 새로 개설되는 반은 반 ID가 서로 다르므로 식별상의 혼동이 발생하지 않으며, 이 규칙이 없으면 한 과목당 누적 26개의 반만 개설할 수 있어 장기 운영이 불가능하기 때문이다.

예를 들어 수학 과목에 "A반"이 이미 존재한다면 같은 수학 과목에 새로운 "A반"을 개설할 수 없다. 그러나 영어 과목에는 별도의 "A반"을 개설할 수 있다.

동치 비교: 두 반 이름의 문자열 전체가 완전히 일치하는 경우에만 서로 동치인 반 이름으로 간주한다.

검색: 화면에 표시할 때에는 과목명과 반 이름을 함께 표시할 수 있다. 예를 들어 수학 과목의 "A반"을 "수학 A반"으로 표시할 수 있으나, 데이터 파일에 저장되는 반 이름은 "A반"이다.

4.3 비밀번호

비밀번호는 사용자 ID와 함께 사용하여 프로그램에 로그인하려는 사용자가 해당 사용자 계정의 사용자인지 확인하기 위한 문자열이다.

문법 형식: 문법적으로 올바른 비밀번호는 다음 조건을 모두 만족하는 문자열이다.

- 길이는 8 이상 20 이하이다.
- 로마자 대문자 "A"부터 "Z", 로마자 소문자 "a"부터 "z", 숫자 "0"부터 "9", 그리고 "!", "@", "#", "\$", "%" 문자를 사용할 수 있다.
- 표준공백, 탭, 개행은 포함할 수 없다.

예를 들어 "academy01", "MinSu123", "study2026!", "class#123"은 모두 올바른 비밀번호이지만, "abc123", "my password", "password_01"은 올바른 비밀번호가 아니다.

"abc123"은 길이가 8 미만이고, "my password"는 표준공백을 포함한다. "password_01"은 허용되지 않는 "_" 문자를 포함한다.

의미 규칙: 비밀번호에는 문법 형식 이외의 추가적인 의미 규칙을 두지 않는다.

동치 비교: 두 비밀번호의 문자열 전체가 완전히 일치하는 경우에만 서로 동치인 것으로 간주한다. 로마자 대소문자를 구분한다.

예를 들어 “Academy01”과 “academy01”은 서로 다른 비밀번호이다.

검색: 비밀번호를 이용한 검색 기능은 제공하지 않는다.

4.4 사용자 역할

사용자 역할은 사용자 계정이 원장, 강사, 학생 중 어느 종류의 사용자인지를 나타내는 값이다. 사용자 역할에 따라 로그인 이후 사용할 수 있는 기능과 접근 가능한 데이터의 범위가 달라진다.

문법 형식: 문법적으로 올바른 사용자 역할은 다음 세 문자열 중 하나이다.

"원장", "강사", "학생"

위 세 문자열 이외의 문자열은 문법적으로 올바른 사용자 역할이 아니다.

의미 규칙: 하나의 사용자 계정은 반드시 하나의 사용자 역할만 가진다.

동치 비교: 두 사용자 역할의 문자열이 완전히 일치하는 경우에만 서로 동치인 역할로 간주한다.

검색: 사용자 역할 자체를 검색어로 사용하는 기능은 제공하지 않는다.

4.5 연락처

연락처는 학생 또는 강사에게 연락하기 위한 전화번호를 나타내는 문자열이다. 학생 데이터와 강사 데이터는 각각 하나의 연락처를 가진다. 기존 초안에서도 학생과 강사의 필드에 연락처를 포함하고 있다.

문법 형식: 문법적으로 올바른 연락처는 다음 조건을 모두 만족하는 문자열이다.

- 숫자 "0"부터 "9"와 "-" 문자만 사용할 수 있다.
- "-" 문자를 모두 제거했을 때 숫자의 개수는 9 이상 11 이하이다.

- 첫 문자와 마지막 문자는 숫자이다.
- "-" 문자가 두 개 이상 연속해서 등장할 수 없다.
- 표준공백, 탭, 개행은 포함할 수 없다.
- "-" 문자는 생략할 수 있다.

예를 들어 “010-1234-5678”, “01012345678”, “02-123-4567”, “021234567”은 모두 올바른 연락처이지만, “010--1234-5678”, “-010-1234-5678”, “010 1234 5678”, “123-456”은 올바른 연락처가 아니다.

첫 번째 문자열에는 연속된 "-"가 있으며, 두 번째 문자열은 "-"로 시작한다. 세 번째 문자열은 표준공백을 포함하고, 마지막 문자열은 "-"를 제거했을 때 숫자의 개수가 9 미만이다.

의미 규칙: 연락처에는 추가적인 의미 규칙을 두지 않는다.

동치 비교: 두 연락처를 비교할 때에는 양쪽 문자열에 포함된 모든 "-" 문자를 제거한 뒤 남은 숫자 문자열 전체가 동일한 경우 서로 동치인 연락처로 간주한다.

따라서 “010-1234-5678”와 “01012345678”는 서로 동치이다. 반면 “010-1234-5678”와 “010-1234-5679”는 서로 동치가 아니다.

학생 또는 강사를 연락처로 검색할 때에는 검색어와 저장된 연락처에서 모든 "-" 문자를 제거한 뒤, 검색어가 저장된 연락처의 부분문자열인 경우 합치된 것으로 간주한다.

예를 들어 검색어가 “12345678”인 경우 다음 연락처에 모두 합치된다.

010-1234-5678

01012345678

마찬가지로 "010-1234"와 "0101234"는 검색 과정에서 동일한 검색어로 취급한다.

4.6 상태

상태는 사용자 학생, 강사 반, 수강 등록이 현재 어떤 상태에 있는지를 나타내는 값이다.

상태의 종류에 따라 허용되는 값이 서로 다르므로 학생 상태, 강사 상태, 반 상태, 수강 등록 상태로 구분한다.

4.6.1 학생 상태

학생 상태는 해당 학생이 현재 학원에 등록되어 있는지를 나타낸다.

문법 형식: 문법적으로 올바른 학생 상태는 다음 두 문자열 중 하나이다.

"재원", "퇴원"

의미 규칙: “재원” 상태의 학생만 새로운 반에 수강 신청할 수 있다. “퇴원” 상태의 학생 정보와 기존 수강 기록은 삭제하지 않고 유지한다.

동치 비교: 두 학생 상태의 문자열 전체가 완전히 일치하는 경우에만 서로 동치인 것으로 간주한다.

검색: 학생 상태를 검색어로 사용하는 기능은 제공하지 않는다.

4.6.2 강사 상태

강사 상태는 해당 강사가 현재 학원에서 수업을 담당할 수 있는 상태인지를 나타낸다.

문법 형식: 문법적으로 올바른 강사 상태는 다음 두 문자열 중 하나이다.

"재직", "퇴직"

의미 규칙: “재직” 상태의 강사만 새로 개설하거나 수정하는 반의 담당 강사로 지정할 수 있다. “퇴직” 상태의 강사 정보와 기존 담당 기록은 삭제하지 않고 유지한다.

동치 비교: 두 강사 상태의 문자열 전체가 완전히 일치하는 경우에만 서로 동치인 것으로 간주한다.

검색: 강사 상태를 검색어로 사용하는 기능은 제공하지 않는다.

4.6.3 반 상태

반 상태는 해당 반의 현재 운영 상태를 나타낸다.

문법 형식: 문법적으로 올바른 반 상태는 다음 문자열 중 하나이다.

"개설", "폐강"

의미 규칙:

- “개설”: 수업이 개설되어 시작 날짜 전에는 수강신청이 가능하며 그 이후에는 수업이 정상적으로 운영되고 있는 상태이다.
- “폐강”: 수업을 더 이상 운영하지 않는 상태이다.

새로운 수강 신청은 "개설" 상태이면서 "가상 현재 일시"의 날짜가 "시작 날짜"보다 이전인 경우에 대해서만 허용한다. 즉, 시작 전날까지 신청을 받으며, 시작 날짜 당일부터는 신청을 받지 않는다.

동치 비교: 두 반 상태의 문자열 전체가 완전히 일치하는 경우에만 서로 동치인 것으로 간주한다.

표시 상태: 조회 화면에 표시하는 상태는 저장된 반 상태와 가상 현재 일시, 시작 날짜, 종료 날짜로부터 계산하며 데이터 파일에 저장하지 않는다.

* 반 상태가 "폐강"이면 폐강으로 표시한다.

* 반 상태가 "개설"이고 가상 현재 일시의 날짜가 시작 날짜보다 이전이면 모집중으로 표시한다.

* 반 상태가 "개설"이고 가상 현재 일시의 날짜가 시작 날짜와 같거나 이후이면서 종료 날짜 이전이거나 같으면 진행중으로 표시한다.

* 반 상태가 "개설"이고 가상 현재 일시가 종료 날짜보다 이후이면 종료로 표시한다.

검색: 반 상태를 검색어로 사용하는 기능은 제공하지 않는다.

4.6.4 수강 등록 상태

수강 등록 상태는 특정 학생과 특정 반 사이의 수강 관계가 현재 유효한지를 나타낸다.

문법 형식: 문법적으로 올바른 수강 등록 상태는 다음 두 문자열 중 하나이다.

"수강중", "취소"

의미 규칙: “수강중” 상태의 수강 등록만 현재 유효한 수강으로 간주한다.

정원 계산, 학생 시간 충돌 검사 등 현재 수강 관계가 필요한 경우 “수강중” 상태의 등록만 계산 대상으로 사용한다.

“취소” 상태의 수강 등록은 삭제하지 않고 기존 기록으로 유지한다.

동치 비교: 두 수강 등록 상태의 문자열 전체가 완전히 일치하는 경우에만 서로 동치인 것으로 간주한다.

조회 화면에 표시하는 수강 등록 상태는 저장된 수강 등록 상태와 해당 반의 표시 상태(4.6.3절)로부터 계산하며 데이터 파일에 저장하지 않는다.

수강 등록 상태가 "취소"이면 취소로 표시한다.

수강 등록 상태가 "수강중"이고 반의 표시 상태가 "종료"이면 수강 완료로 표시한다.

그 외의 경우("수강중"이면서 반의 표시 상태가 "모집중" 또는 "진행중")에는 수강중으로 표시한다.

검색: 수강 등록 상태를 검색어로 사용하는 기능은 제공하지 않는다.

4.7 날짜

날짜는 반의 시작일과 종료일 등 연·월·일 단위의 시점을 나타내는 데이터이다.

문법 형식: 문법적으로 올바른 날짜는 다음 형식에 부합하는 문자열이다.

YYYY-MM-DD

각 부분은 다음과 같이 구성한다.

- YYYY: 숫자 4개
- MM: 숫자 2개
- DD: 숫자 2개
- 연·월·일 사이에는 각각 “-” 문자 하나를 사용한다.

예를 들어 “2026-09-27”, “2026-01-01”, “2030-12-31”은 모두 올바른 날짜이지만, “26-09-27”, “2026/09/27”, “2026-9-27”, “20260927”은 올바른 날짜가 아니다.

해석: 첫 네 숫자를 연도, 다음 두 숫자를 월, 마지막 두 숫자를 일로 해석한다.

의미 규칙: 문법적으로 올바른 날짜는 실제 그레고리력에 존재하는 날짜여야 한다.

예를 들어 “2026-02-30”은 실제 존재하지 않는 날짜이며, “2026-13-01”은 월의 범위를 벗어나므로 문법적으로는 올바르지만 의미상 올바르지 않다.

동치 비교: 두 날짜를 해석한 연도, 월, 일이 모두 같은 경우에만 서로 동치인 날짜로 간주한다.

순서 비교: 두 날짜의 연도, 월, 일 순서에 따라 한 날짜가 다른 날짜보다 이전인지, 같은 날인지, 이후인지를 비교한다.

검색: 날짜를 부분 검색하는 기능은 제공하지 않는다.

4.8 시각

시각은 하루 안에서 시와 분 단위의 시간을 나타내는 데이터이다. 반의 수업 시작 시각과 종료 시각 등에 사용한다.

문법 형식: 문법적으로 올바른 시각은 다음 형식에 부합하는 문자열이다.

HH:MM

각 부분은 다음과 같이 구성한다.

- HH: 숫자 2개
- MM: 숫자 2개
- 시와 분 사이에는 “:” 문자 하나를 사용한다.

해석: 앞의 두 숫자를 시, 뒤의 두 숫자를 분으로 해석한다.

의미 규칙: 문법적으로 올바른 시각은 다음 조건을 모두 만족해야 한다.

- 시는 00 이상 23 이하이다.
- 분은 00 이상 59 이하이다.

예를 들어 “09:00”, “14:30”, “23:59”는 모두 올바른 시각이지만, “24:00”, “12:60”은 올바른 시각이 아니다.

동치 비교: 두 시각을 해석한 시와 분이 모두 같은 경우에만 서로 동치인 시각으로 간주한다.

순서 비교: 시가 작은 시각을 더 이른 시각으로 간주하며, 시가 같으면 분이 작은 시각을 더 이른 시각으로 간주한다.

검색: 시각을 부분 검색하는 기능은 제공하지 않는다.

4.9 일시

일시는 하나의 날짜와 하나의 시각을 결합하여 특정 시점을 나타내는 데이터이다.

문법 형식: 문법적으로 올바른 일시는 다음 형식에 부합하는 문자열이다.

〈날짜〉_〈시각〉

예를 들어 “2026-09-27 14:30”은 올바른 일시의 예이다.

해석: 앞부분은 4.7절의 날짜 규칙에 따라 날짜로 해석하고, 뒷부분은 4.8절의 시각 규칙에 따라 시각으로 해석한다.

의미 규칙: 일시에 포함된 날짜와 시각은 각각 자신의 의미 규칙을 만족해야 한다.

동치 비교: 날짜와 시각이 모두 동치인 경우에만 두 일시를 서로 동치인 것으로 간주한다.

순서 비교: 날짜가 빠른 일시를 더 이른 일시로 간주한다. 날짜가 같은 경우에는 시각이 빠른 일시를 더 이른 일시로 간주한다.

예를 들어 “2026-09-27 09:00”은 “2026-09-27 10:00”보다 이른 일시이고, “2026-09-27 10:00”은 “2026-09-28 09:00”보다 이른 일시이다.

검색: 일시를 검색어로 사용하는 기능은 제공하지 않는다.

4.10 교시

교시는 학원에서 수업이 진행되는 시간대를 구분하기 위한 60분 단위의 데이터 요소이다. 교시는 1교시부터 10교시까지 존재하며, 각 교시는 서로 연속된 시간 구간을 나타낸다.

문법 형식: 문법적으로 올바른 교시는 다음 조건을 모두 만족하는 문자열이다.

- 숫자 0부터 9만으로 이루어진다.
- 길이는 1 이상 2 이하이다.
- 선행 0을 사용할 수 없다.

예를 들어 "1", "5", "10"은 문법적으로 올바른 교시이다. "01", "1교시", "+1"은 문법적으로 올바른 교시가 아니다.

단, "0", "11", "99"는 위 문법 형식에는 부합하지만 아래의 의미 규칙을 만족하지 않으므로 의미상 올바른 교시는 아니다.

해석: 교시 문자열을 십진수 정수로 해석한다.

각 교시에 대응하는 실제 수업 시간은 다음과 같다.

교시	시작 시각	종료 시각
1교시	09:00	10:00
2교시	10:00	11:00
3교시	11:00	12:00
4교시	12:00	13:00
5교시	13:00	14:00

6교시 14:00 15:00

7교시 15:00 16:00

8교시 16:00 17:00

9교시 17:00 18:00

10교시 18:00 19:00

의미 규칙: 문법적으로 올바른 교시를 십진수 정수로 해석한 값은 1 이상 10 이하여야 한다.

따라서 "1", "5", "10"은 의미상 올바른 교시이고, "0"과 "11"은 문법적으로는 올바르지만 의미상 올바른 교시는 아니다.

교시는 반의 수업 일정을 나타낼 때 사용한다. 반의 수업 시간은 임의의 시각을 직접 지정하지 않고, 시작 교시와 종료 교시를 이용하여 나타낸다. 시작 교시와 종료 교시 사이의 관계와 여러 반의 수업 일정이 충돌하는 조건은 4.12 수업 일정에서 정의한다.

동치 비교: 두 교시를 십진수 정수로 해석한 값이 같은 경우에만 서로 동치인 교시로 간주한다.

현재 문법에서는 선행 0을 허용하지 않으므로, 문법적·의미적으로 올바른 두 교시가 동치라면 두 문자열도 동일하다.

순서 비교: 십진수 정수로 해석한 값이 작은 교시를 더 이른 교시로 간주한다.

검색: 교시 자체를 검색어로 사용하는 기능은 제공하지 않는다.

4.11 요일

요일은 반복되는 수업이 한 주의 어느 날에 진행되는지를 나타내는 데이터이다.

문법 형식: 문법적으로 올바른 요일은 다음 일곱 문자열 중 하나이다.

"월", "화", "수", "목", "금", "토", "일"

의미 규칙: 요일에는 추가적인 의미 규칙을 두지 않는다.

동치 비교: 두 요일의 문자열이 완전히 일치하는 경우에만 서로 동치인 요일로 간주한다.

검색: 요일을 검색어로 사용하는 기능은 제공하지 않는다.

4.11.1 요일 목록

요일 목록은 하나의 반이 일주일에 둘 이상의 요일에 수업을 진행하는 경우 그 수업 요일들을 함께 나타내기 위한 데이터이다.

문법 형식: 문법적으로 올바른 요일 목록은 하나 이상의 요일을 표준공백 한 개로 구분하여 나열한 문자열이다.

예를 들어 “월”, “월 수”, “화 목”, “월 수 금”, “월 수 월”은 모두 문법적으로 올바른 요일 목록이다.

의미 규칙: 문법적으로 올바른 요일 목록은 다음 조건을 모두 만족해야 한다.

- 하나의 요일 목록에는 서로 동치인 요일이 두 번 이상 포함될 수 없다.
- 요일 목록에는 최소 하나 이상의 요일이 포함되어야 한다.

따라서 “월 수 월”은 문법적으로는 올바르지만, “월”이 두 번 포함되어 있으므로 의미상 올바른 요일 목록이 아니다.

표준형: 요일은 다음 순서로 저장한다.

“월 화 수 목 금 토 일”

사용자가 요일을 다른 순서로 입력하더라도 저장할 때에는 위 순서에 맞추어 정렬한다.

예를 들어 “금 월 수”를 입력한 경우 저장되는 요일 목록의 표준형은 “월 수 금”이다.

동치 비교: 두 요일 목록에 포함된 요일의 종류가 모두 같은 경우 나열된 순서와 관계없이 서로 동치인 요일 목록으로 간주한다.

예를 들어 “월 수 금”과 “금 월 수”는 서로 동치인 요일 목록이다. 다만 데이터 파일에 저장할 때에는 두 경우 모두 표준형인 “월 수 금”으로 저장한다.

검색: 요일 목록 자체를 검색어로 사용하는 기능은 제공하지 않는다.

4.12 수업 일정

수업 일정은 하나의 반이 어느 기간 동안, 어떤 요일에, 어느 교시부터 어느 교시까지 수업을 진행하는지를 나타내는 복합 데이터이다.

하나의 수업 일정은 다음 데이터 요소로 구성된다.

〈시작 날짜〉

〈종료 날짜〉

〈요일 목록〉

〈시작 교시〉

〈종료 교시〉

예를 들어 “시작 날짜: 2026-09-01, 종료 날짜: 2026-12-20, 요일 목록: 월 수, 시작 교시: 1, 종료 교시: 3”은 올바른 수업 일정이지만, “시작 날짜: 2026-12-20, 종료 날짜: 2026-09-01” 또는 “시작 교시: 3, 종료 교시: 1”은 올바른 수업 일정이지 않다.

문법 형식: 수업 일정을 구성하는 각 데이터 요소는 각각 4.7절 날짜, 4.10절 교시, 4.11.1절 요일 목록의 문법 형식을 만족해야 한다.

의미 규칙: 하나의 수업 일정은 다음 조건을 모두 만족해야 한다.

- 시작 날짜는 종료 날짜보다 이전이거나 같아야 한다.
- 시작 교시는 종료 교시보다 이전이거나 같아야 한다.
- 요일 목록은 4.11.1절의 의미 규칙을 만족해야 한다.
- 시작 교시부터 종료 교시까지의 모든 교시는 해당 수업 일정이 사용하는 것으로 본다.
- 시작 날짜의 요일은 요일 목록에 포함되어야 한다. 즉, 시작 날짜는 해당 수업 일정의 첫 수업일(개강일)과 일치한다.
- 종료 날짜의 요일은 요일 목록에 포함되어야 한다. 즉, 종료 날짜는 해당 수업 일정의 마지막 수업일(종강일)과 일치한다.

예를 들어 요일 목록이 "월 수"인 경우, 시작 날짜는 월요일 또는 수요일이어야 하고, 종료 날짜도 월요일 또는 수요일이어야 한다. "시작 날짜: 2026-09-01(화), 종료 날짜: 2026-12-20, 요일 목록: 월 수"는 시작 날짜의 요일(화)이 요일 목록에 포함되지 않으므로 올바른 수업 일정이지 않다.

동치 비교: 두 수업 일정의 시작 날짜, 종료 날짜, 요일 목록, 시작 교시와 종료 교시가 각각 모두 동치인 경우에만 두 수업 일정을 서로 동치인 것으로 간주한다.

4.12.1 일정 충돌

두 수업 일정은 다음 조건을 모두 만족하는 경우 서로 충돌하는 것으로 간주한다.

1. 두 수업 기간이 겹치는 구간이 존재한다.
2. 두 수업 기간이 겹치는 구간 안에, 두 요일 목록에 공통으로 포함된 요일에 해당하는 날짜가 하나 이상 존재한다.
3. 두 수업이 사용하는 교시가 하나 이상 겹친다.

두 수업 일정의 기간이 각각 **A 시작 날짜 ~ A 종료 날짜**, **B 시작 날짜 ~ B 종료 날짜**인 경우, 두 기간의 겹치는 구간은 두 시작 날짜 중 더 늦은 날짜부터 두 종료 날짜 중 더 이른 날짜까지로 본다. 이때 더 늦은 시작 날짜가 더 이른 종료 날짜보다 이후라면 두 기간은 겹치지 않는다.

교시의 경우 시작 교시부터 종료 교시까지의 모든 교시를 해당 수업이 사용하는 것으로 본다. 따라서 두 수업의 교시 범위에 하나 이상의 동일한 교시가 포함되어 있으면 교시가 겹치는 것으로 간주한다.

예를 들어 다음 두 수업은 3교시를 공통으로 사용하므로, 수업 기간과 요일 조건도 만족한다면 서로 충돌한다.

수업 A: 1교시 ~ 3교시

수업 B: 3교시 ~ 5교시

반면 다음 두 수업은 사용하는 교시가 서로 겹치지 않으므로 충돌하지 않는다.

수업 A: 1교시 ~ 3교시

수업 B: 4교시 ~ 6교시

즉, 한 수업의 종료 교시 바로 다음 교시에 다른 수업이 시작하는 경우에는 서로 겹치지 않는 것으로 간주한다.

예를 들어 다음 두 일정은 서로 충돌한다.

수업 A

기간: 2026-09-01 ~ 2026-12-20

요일 목록: 월 수

교시: 1교시 ~ 3교시

수업 B

기간: 2026-10-01 ~ 2026-11-30

요일 목록: 수 금

교시: 3교시 ~ 5교시

두 수업의 기간이 겹치고, 그 겹치는 기간 안에 두 수업이 공통으로 진행되는 수요일이 존재하며, 3교시를 공통으로 사용하므로 두 일정은 충돌한다.

반면 다음 두 일정은 교시가 겹치더라도 충돌하지 않는다.

수업 A

기간: 2026-09-01 ~ 2026-12-20

요일 목록: 월

교시: 1교시 ~ 3교시

수업 B

기간: 2026-09-01 ~ 2026-12-20

요일 목록: 화

교시: 2교시 ~ 4교시

두 수업의 기간과 교시는 겹치지만 공통된 수업 요일이 없으므로 충돌하지 않는다.

두 수업의 기간과 교시는 겹치지만 공통된 수업 요일이 없으므로 충돌하지 않는다.

동치 비교: 두 수업 일정의 시작 날짜, 종료 날짜, 요일 목록, 시작 교시, 종료 교시가 각각 모두 동치인 경우에만 두 수업 일정을 서로 동치인 것으로 간주한다.

4.13 정원

정원은 하나의 반에 동시에 수강 등록할 수 있는 학생의 최대 인원을 나타내는 양의 정수이다.

현재 수강 인원은 정원과 별도의 저장 데이터로 두지 않고, 해당 반에 연결된 “수강중” 상태의 수강 등록 개수를 이용하여 계산한다.

문법 형식: 문법적으로 올바른 정원은 다음 조건을 모두 만족하는 문자열이다.

- 숫자 “0”부터 “9”만으로 이루어진다.
- 길이는 1 이상 3 미만이다.
- 선행 0은 허용하지 않는다. 단, 문자열 “0” 자체는 문법적으로 허용한다.

예를 들어 “0”, “1”, “10”, “25”은 모두 문법적으로 올바른 정원이지만, “01”, “0010”, “10명”, “+10”은 문법적으로 올바른 정원이 아니다.

해석: 문자열을 10진수 정수로 해석한다.

의미 규칙: 의미상 올바른 정원은 1 이상 99 이하의 정수이어야 한다.

따라서 “0”은 문법적으로는 올바르지만 의미상 올바른 정원은 아니다.

동치 비교: 두 정원을 10진수 정수로 해석한 값이 같은 경우 서로 동치인 정원으로 간주한다.

현재 문법에서는 선행 0을 허용하지 않으므로, 문법적으로 올바른 두 정원이 동치라면 실제 문자열도 동일하게 된다.

정원 초과 판단: 특정 반에 대해 “수강중” 상태인 수강 등록의 개수가 정원과 같거나 큰 경우 새로운 수강 등록을 허용하지 않는다.

예를 들어 정원이 10명이고 현재 “수강중” 상태의 수강 등록이 10건인 경우 새로운 학생은 해당 반을 수강 신청할 수 없다.

검색: 정원을 검색어로 사용하는 기능은 제공하지 않는다.

4.14 가상 현재 일시

가상 현재 일시는 프로그램이 현재 시점, 즉 “지금”으로 간주하는 일시이다.

본 프로그램은 실제 컴퓨터의 날짜와 시각을 사용하지 않고, `data/datetime.txt`에 저장된 가상 현재 일시를 프로그램 전체의 기준 시점으로 사용한다. 수강 신청 가능 여부, 반의 수업 기간 시작 여부 등 현재 시점에 따라 결과가 달라지는 기능은 모두 이 가상 현재 일시를 기준으로 판단한다.

이를 통해 실제 날짜가 바뀌기를 기다리지 않고 임의의 시점에서 시간에 따른 프로그램의 동작을 확인할 수 있다.

역행 판정 기준 일시는 아래 "의미 규칙"에서 정의하는 "기록 일시" 집합의 최댓값과 현재 저장된 가상 현재 일시 중 더 나중의 일시를 말한다. 새 가상 현재 일시가 이 기준 일시보다 이전이면 변경을 거부하며, 동일한 일시는 허용한다.

문법 형식: 가상 현재 일시는 4.9절의 일시와 동일한 데이터 형식을 사용한다.

따라서 가상 현재 일시는 다음 형식의 문자열이다.

〈날짜〉 _ 〈시각〉

예를 들어 다음은 올바른 가상 현재 일시의 표현이다.

2026-09-27 14:30

가상 현재 일시에 포함되는 날짜는 4.7절 날짜, 시각은 4.8절 시각의 문법 형식과 의미 규칙을 각각 만족해야 하며, 전체 일시는 4.9절 일시의 규칙을 만족해야 한다.

따라서 가상 현재 일시만을 위한 별도의 날짜 또는 시각 문법은 정의하지 않는다.

의미 규칙: 문법적으로 올바른 가상 현재 일시는 다음 조건을 모두 만족해야 한다.

- 가상 현재 일시는 아래에 정의한 "기록 일시" 집합의 모든 값 중 가장 나중의 일시보다 이전일 수 없다. 본 절에서 "기록 일시"란 프로그램에서 이미 발생한 사건의 시점을 가리키며, 다음 세 종류만을 의미한다.
 - ① enrollments.txt의 수강 등록 일시
 - ② enrollments.txt의 수강 취소 일시(취소 상태인 레코드에 한함)
 - ③ 과거에 지정된 적이 있는 모든 가상 현재 일시(가상 현재 일시 이력, 아래 "저장" 항목 참조)

다음은 "기록 일시"에 포함하지 않는다. 이들은 "예정된 미래 시점"이거나 사건의 발생 시점이 아니기 때문이다.

classes.txt의 수업 시작일·종료일·수업 시간 (아직 발생하지 않은 미래의 예정)

users.txt·students.txt·teachers.txt·subjects.txt의 등록 시각 등 참조용 메타데이터

- 실행 중 가상 현재 일시를 변경하는 경우, 새 가상 현재 일시는 변경 직전의 가상 현재 일시보다 이전일 수 없다.
- 이전과 동일한 일시로 지정하는 것은 허용한다.

일시의 이전·동일·이후 관계는 4.9절 일시의 순서 비교 규칙에 따른다.

예를 들어 현재 저장된 가상 현재 일시가

2026-09-27 14:30

인 경우 다음 값으로 변경하는 것은 허용한다.

2026-09-27 14:30

2026-09-27 15:00

2026-09-28 09:00

반면 다음 값으로 변경하는 것은 허용하지 않는다.

2026-09-27 14:29

또한 데이터에 저장되어 있는 가장 나중의 기록 일시가

2026-10-03 13:00

이라면, 가상 현재 일시는 2026-10-03 13:00보다 이전일 수 없다.

예를 들어 사용자가 `datetime.txt`의 내용을 직접 수정하여

2026-09-01 09:00

으로 변경하였더라도, 이미 수강 신청 등의 기록 중 2026-10-03 13:00에 해당하는 기록이 존재한다면 해당 가상 현재 일시는 의미상 올바르지 않다.

이 규칙은 이미 발생한 기록보다 프로그램의 현재 시점이 과거가 되는 것을 방지하기 위한 것이다.

미래 예정 시점은 비교 대상이 아님 반면 `classes.txt`에 수업 시작일 2026-12-01인 반이 등록되어 있더라도, 이는 아직 발생하지 않은 미래의 예정 시점이므로 하한선 판정에 사용하지 않는다. 즉, 현재 가상 현재 일시가 2026-10-10 09:00인 상태는 올바르며, 수업 시작일 때문에 "12월 이전으로 시간을 둘 수 없다"고 판단하지 않는다.

업무 기록이 없는 시간 전진에 대한 보호: 단, 위 규칙만으로는 수강 신청·취소 등의 업무 기록이 전혀 발생하지 않은 상태에서 가상 현재 일시만 전진한 경우를 탐지할 수 없다. 예를 들어 사용자가 가상 현재 일시를 2026-09-27 14:30에서 2026-10-15 09:00으로 전진시킨 뒤, 업무 기록이 전혀 없는 상태에서 `datetime.txt`를 2026-09-27 14:30으로 되돌려 저장하였다고 하자. 업무 기록만을 기준으로 하면 이 변경은 탐지되지 않지만, 가상 현재 일시 이력에 2026-10-15 09:00이 남아 있으므로 의미상 올바르지 않은 가상 현재 일시로 판단한다.

지정 및 변경: 가상 현재 일시는 다음 두 경우에 지정한다.

1. 프로그램을 처음 실행하여 `datetime.txt`가 존재하지 않는 경우
2. 원장이 로그인한 뒤 가상 현재 일시 변경 기능을 선택한 경우

첫 번째 경우는 2.3.1절의 최초 실행 절차에 해당한다. 기존 가상 현재 일시가 없으므로 입력을 생략할 수 없으며, 반드시 올바른 가상 현재 일시를 하나 입력해야 한다. 이 단계는 원장 계정을 생성하는 중이므로 로그인 이전이며, 권한 검사의 예외로 본다.

최초 생성 이후의 실행에서는 **datetime.txt**에 저장된 값을 그대로 사용하며, 프로그램 시작 시 새로운 가상 현재 일시를 입력받지 않는다.

두 번째 경우는 원장만 수행할 수 있으며, 상세한 화면과 절차는 6.3절에서 정의한다. 새로운 일시를 입력한 경우에는 위의 문법 규칙과 의미 규칙을 모두 만족해야 변경할 수 있다.

동일 값 재지정 처리 가상 현재 일시를 변경 직전의 값과 동일한 값으로 재지정하는 것은 허용하며, 이 경우 이력에 중복해서 기록하지 않는다. 이력의 목적은 "과거에 도달했던 가장 나중 시점"을 보존하는 것이므로, 동일 값 중복은 하한선 판정에 영향을 주지 않는다.

시간의 진행: 가상 현재 일시는 프로그램 실행 중에 자동으로 증가하지 않는다.

명령을 실행하거나 실제 시간이 경과하더라도 가상 현재 일시는 변하지 않으며, 사용자가 명시적으로 변경한 경우에만 값이 변경된다. 따라서 동일한 가상 현재 일시와 동일한 데이터 상태에서는 실제 프로그램 실행 시간이 달라지더라도 시간과 관련된 판단 결과는 동일하다.

전진 사실의 영구 보존 다만 사용자가 가상 현재 일시를 명시적으로 전진시킨 사실 자체는 영구적으로 보존된다. 수강 신청·취소 등의 업무 기록이 전혀 발생하지 않은 시간 전진이라 하더라도 아래 "저장" 규칙에 따라 이력에 남으므로, 이후에 그보다 과거의 일시로 되돌릴 수 없다.

동치 비교: 두 가상 현재 일시의 동치 비교는 4.9절 일시의 동치 비교 규칙을 따른다.

즉, 두 가상 현재 일시에 포함된 날짜와 시각이 각각 모두 동치인 경우 두 가상 현재 일시를 서로 동치인 것으로 간주한다.

순서 비교: 두 가상 현재 일시의 순서 비교는 4.9절 일시의 순서 비교 규칙을 따른다.

따라서 날짜가 빠른 일시가 더 이전이며, 날짜가 같은 경우에는 시각이 빠른 일시가 더 이전이다.

저장: 가상 현재 일시와 그 변경 이력은 **data/datetime.txt**에 함께 저장한다. 파일에는 한 줄에 하나씩 4.9절 일시 형식의 값을 저장하며, 각 줄의 역할은 다음과 같다.

파일의 마지막 줄은 현재 유효한 가상 현재 일시이다.

그 이전 줄들은 과거에 지정되었던 가상 현재 일시의 이력이며, 지정된 순서(오래된 것부터 최근 것 순)대로 저장한다.

가상 현재 일시가 새로 지정되거나 변경될 때마다 파일 끝에 새 줄을 추가하는 방식으로 즉시 저장한다. 기존 줄은 수정하거나 삭제하지 않는다.

이력 줄은 사용자가 임의로 삭제·수정할 수 없다. 이를 어긴 경우 **5.4절** 무결성 확인 및 처리에서 오류로 처리한다.

구체적으로, 다음 중 하나라도 해당하면 "이력 줄이 임의로 삭제·수정된 것"으로 간주한다.

- ① 파일의 어느 줄이 **4.9절** 일시 형식을 만족하지 않는 경우
- ② 파일의 줄들이 지정 순서(비내림차순)로 정렬되어 있지 않은 경우, 즉 앞 줄의 일시가 뒤 줄의 일시보다 나중인 경우
- ③ 파일의 마지막 줄(현재 가상 현재 일시)이 앞선 이력 줄들 중 어느 하나보다도 이전인 경우

예를 들어 사용자가 가상 현재 일시를 2026-09-27 14:30 -> 2026-09-28 09:00 -> 2026-10-15 09:00 순으로 지정하였다면, **datetime.txt**의 전체 내용은 다음과 같다.

2026-09-27 14:30

2026-09-28 09:00

2026-10-15 09:00

이 중 마지막 줄 **2026-10-15 09:00**이 현재 유효한 가상 현재 일시이며, 앞의 두 줄은 이력이다. 가상 현재 일시의 하한선을 판정할 때는 이 파일에 기록된 모든 일시(이력 포함)와 위 "의미 규칙"에서 정의한 ①·② 업무 기록 일시 전체를 함께 고려한다

가상 현재 일시가 변경되면 변경된 값을 즉시 **datetime.txt**에 저장한다. 따라서 프로그램을 종료한 뒤 다시 실행하더라도 마지막으로 지정한 가상 현재 일시가 유지된다.

4.15 수강 등록

수강 등록은 특정 학생과 특정 반 사이에 수강 관계가 성립한 것을 나타내는 복합 데이터 요소이다. 하나의 수강 등록은 하나의 학생과 하나의 반을 연결하며, 수강 신청이 이루어진 일시와 현재 수강 관계의 상태를 함께 가진다.

하나의 수강 등록은 다음 데이터 요소로 구성된다.

- 수강 등록 ID

- 학생 ID
- 반 ID
- 등록 일시
- 수강 등록 상태
- 취소 일시

각 구성 요소는 다음과 같은 기존 데이터 요소의 규칙을 따른다.

- 수강 등록 ID: 4.1절의 내부 식별자 규칙을 따른다.
- 학생 ID: 4.1절의 내부 식별자 규칙을 따르며, 학생 데이터를 참조한다.
- 반 ID: 4.1절의 내부 식별자 규칙을 따르며, 반 데이터를 참조한다.
- 등록 일시: 4.9절의 일시 규칙을 따른다.
- 수강 등록 상태: 4.6.4절의 수강 등록 상태 규칙을 따른다.
- 취소 일시: 4.9절의 일시 규칙을 따른다. 수강 중인 등록에서는 취소 일시를 갖지 않는다.

의미 규칙

수강 등록은 다음 조건을 만족해야 한다.

- 학생 ID가 가리키는 학생 데이터가 실제로 존재해야 한다.
- 반 ID가 가리키는 반 데이터가 실제로 존재해야 한다.
- 하나의 수강 등록은 하나의 학생과 하나의 반만 연결한다.
- 동일한 학생과 동일한 반에 대해 동시에 둘 이상의 "수강중" 상태 수강 등록이 존재할 수 없다.
- 수강 등록 상태가 "수강중"인 경우 취소 일시는 존재하지 않는다.
- 수강 등록 상태가 "취소"인 경우 취소 일시는 올바른 일시여야 하며, 등록 일시보다 이전일 수 없다.
- 수강 취소가 이루어진 뒤에도 해당 수강 등록 데이터 자체는 삭제하지 않고 취소 상태로 유지한다.
- 학생은 본인의 수강중 등록을 기간 제한 없이 취소할 수 있다. 원장은 모든 학생의 수강중 등록을 기간 제한 없이 취소할 수 있다.
- 이미 취소된 등록은 다시 취소할 수 없다. 취소 일시는 가상 현재 일시로 기록하며 기존 등록 기록은 삭제하지 않는다.

- 취소된 반에 다시 신청하는 경우 최초 신청과 동일한 조건을 검사한다. 신청 가능 기간 안이고 모든 조건을 만족하면 새로운 수강 등록 ID를 부여하여 등록을 생성한다. 기존 취소 기록은 변경하거나 삭제하지 않는다. 신청 가능 기간이 지난 경우에는 재신청할 수 없다.

정원 초과 여부, 학생의 다른 수강과의 시간 충돌 여부, 수강 신청 가능 기간 등은 수강 등록 데이터 자체의 형식이 아니라 수강 신청 과정에서 다른 데이터와 함께 판단하는 조건으로 본다.

동치 비교

두 수강 등록은 수강 등록 ID가 서로 동치인 경우 같은 수강 등록으로 간주한다.

학생 ID와 반 ID가 같더라도 수강 등록 ID가 다르면 서로 다른 수강 등록으로 구분한다. 단, 동일한 학생과 동일한 반에 대해 둘 이상의 "수강중" 상태 등록을 동시에 유지하는 것은 허용하지 않는다.

검색

수강 등록 전체를 문자열의 부분 검색으로 조회하는 기능은 제공하지 않는다.

특정 수강 등록을 지정해야 하는 경우에는 수강 등록 ID를 사용하며, 입력한 수강 등록 ID와 저장된 수강 등록 ID가 동치인 경우에만 해당 등록을 지정한 것으로 간주한다.

5. 데이터 파일

이 프로그램은 사용자 계정, 학생, 강사, 과목, 반, 수강 등록, 가상 현재 일시와 관련된 데이터를 각각 별도의 텍스트 파일에 저장한다.

프로그램에서 사용하는 데이터 파일은 다음과 같다.

- **users.txt**: 사용자 계정 데이터
- **students.txt**: 학생 데이터

- `teachers.txt`: 강사 데이터
- `subjects.txt`: 과목 데이터
- `classes.txt`: 반 데이터
- `enrollments.txt`: 수강 등록 데이터
- `datetime.txt`: 가상 현재 일시 데이터

사용자는 일반적인 텍스트 편집기를 이용하여 데이터 파일의 내용을 직접 확인하거나 수정할 수 있다. 단, 사용자가 직접 생성하거나 수정한 데이터도 이 장에서 정의한 문법 규칙과 의미 규칙을 만족해야 한다.

프로그램이 실행되고 있는 동안 사용자가 외부 텍스트 편집기로 데이터 파일을 수정하는 경우에는 정상적인 동작을 보장하지 않는다.

5.1 공통 규칙

`users.txt`, `students.txt`, `teachers.txt`, `subjects.txt`, `classes.txt`, `enrollments.txt`에는 다음의 공통 규칙을 적용한다. `datetime.txt`는 다른 파일과 구조가 다르므로 5.2.7절에서 별도로 정의한다.

각 데이터 파일의 데이터 레코드 수는 해당 내부 식별자로 표현할 수 있는 범위를 초과할 수 없다.

학생·강사·과목·반·수강 등록 데이터는 각각 최대 10,000개의 서로 다른 ID를 표현할 수 있으므로, 이를 초과하는 레코드는 허용하지 않는다.

5.1.1 인코딩

모든 데이터 파일은 BOM이 없는 UTF-8 인코딩으로 저장한다.

5.1.2 헤더

`users.txt`, `students.txt`, `teachers.txt`, `subjects.txt`, `classes.txt`, `enrollments.txt`의 첫 번째 행은 각 필드의 이름을 나타내는 헤더이다.

헤더는 데이터 레코드로 간주하지 않으며, 프로그램은 두 번째 행부터 데이터 레코드로 읽는다. 헤더의 필드 개수, 필드 이름, 필드 순서는 해당 파일에서 정의한 헤더와 정확히 일치해야 한다. 필드 이름의 오타자, 누락, 추가, 순서 변경, 빈 필드, 선행·후행 표준공백은 허용하지 않는다. 첫 번째 행이 빈 행이거나 헤더 자체가 없는 파일도 문법적으로 올바르지 않다. 단, 정확한 헤더만 존재하고 데이터 레코드가 하나도 없는 파일은 허용한다.

모든 데이터 파일은 5.1.1절에 따라 BOM이 없는 UTF-8이어야 한다. 파일 시작에 BOM이 존재하는 경우 이를 헤더의 일부로 인정하거나 자동으로 제거하지 않고 파일 형식 오류로 처리한다.

`datetime.txt`는 예외로 헤더를 사용하지 않는다.

5.1.3 레코드

헤더를 제외한 하나의 행은 하나의 레코드를 나타낸다.

각 레코드는 해당 파일에서 정한 여러 개의 필드로 구성된다. 하나의 레코드가 둘 이상의 행에 걸쳐 작성될 수 없다.

5.1.4 필드 구분

하나의 레코드 안에서 서로 다른 두 필드 사이에는 정확히 하나의 탭 문자를 사용한다.

이 문서에서는 실제 탭 문자를 시각적으로 구별하기 위해 "`<TAB>`"으로 표시한다. "`<TAB>`"이라는 문자열 자체를 데이터 파일에 저장하는 것이 아니라 해당 위치에 하나의 탭 문자를 사용한다.

예를 들어 세 개의 필드 "A", "B", "C"는 다음과 같이 나타낸다.

`A<TAB>B<TAB>C`

5.1.5 필드의 개수

각 레코드는 해당 데이터 파일에서 정의한 개수의 필드를 정확히 가져야 한다.

필드가 하나라도 부족하거나 필요 이상의 필드가 존재하는 레코드는 문법적으로 올바른 레코드가 아니다.

예를 들어 세 개의 필드를 가져야 하는 레코드에서 "A<TAB>B"와 "A<TAB>B<TAB>C<TAB>D"는 모두 올바른 레코드가 아니다.

5.1.6 빈 줄

데이터 파일의 레코드 사이에 빈 줄을 둘 수 없다. 빈 줄이 존재하는 파일은 문법적으로 올바른 데이터 파일이 아니다.

단, 데이터 레코드가 하나도 없어 헤더만 존재하는 파일은 허용한다. 이 경우 해당 종류의 데이터가 하나도 존재하지 않는 것으로 해석한다.

5.1.7 공백

각 필드의 선행·후행 표준공백을 프로그램이 자동으로 제거하지 않는다.

4장에서 해당 데이터 요소에 선행·후행 공백을 허용하지 않은 경우 공백이 포함된 필드는 문법적으로 올바르지 않다.

예를 들어 "김민수"가 올바른 사람 이름이라도 " 김민수"와 "김민수 "는 올바른 사람 이름 필드가 아니다.

5.1.8 빈 값

각 데이터 파일에서 값이 없는 것이 허용된 필드는 값을 비워 두지 않고 하이픈 문자 '-'를 기록한다. '-'는 해당 필드에 저장된 값이 없음을 나타내는 특수한 표기이며, 실제 필드 값으로 해석하지 않는다.

따라서 탭 문자 사이에 아무 문자도 기록하지 않은 빈 필드는 허용하지 않는다. 단, 각 데이터 요소에서 반드시 값이 존재해야 한다고 정의한 필드에는 -를 사용할 수 없다.

예를 들어 세 번째 필드의 값이 없는 경우 다음과 같이 기록한다.

A<TAB>B<TAB>-<TAB>D

다음과 같이 필드를 실제로 비워 두는 것은 올바르지 않다.

A<TAB>B<TAB><TAB>D

5.1.9 주석

데이터 파일에는 주석을 사용할 수 없다.

따라서 "#", "//" 등으로 시작하는 행에도 별도의 주석 의미를 부여하지 않는다.

5.1.10 마지막 개행

데이터 파일의 마지막 레코드 뒤에는 개행이 있어도 되고 없어도 된다. 두 경우 모두 같은 데이터 파일로 처리한다.

5.1.11 레코드 순서

데이터 파일의 레코드는 내부 식별자나 다른 필드의 값에 따라 정렬되어 있을 필요가 없다.

5.1.12 자동 수정

프로그램은 데이터 파일을 읽는 과정에서 문법 또는 의미 규칙에 위배되는 값을 임의로 수정하지 않는다.

예를 들어 잘못 저장된 학생 ID "S001"을 프로그램이 자동으로 "S0001"로 수정하지 않는다.

데이터 파일에 오류가 발견된 경우에는 5.4절의 무결성 확인 및 처리 규칙에 따라 오류를 출력하며, 별도로 명시된 경우를 제외하고 프로그램이 임의로 값을 변경하지 않는다..

5.1.13 내부 식별자의 부여

학생 ID, 강사 ID, 과목 ID, 반 ID, 수강 등록 ID는 프로그램이 새로운 데이터를 생성할 때 부여한다.

내부 식별자는 4.1.2절에서 정의한 다음 형식을 사용한다.

S0001: 학생 ID

T0001: 강사 ID

P0001: 과목 ID

C0001: 반 ID

E0001: 수강 등록 ID

새로운 내부 식별자를 부여할 때에는 해당 종류의 데이터 파일에 현재 존재하는 내부 식별자들의 숫자 부분 중 가장 큰 값에 1을 더한 값을 사용한다. 해당 종류의 데이터가 하나도 존재하지 않는 경우에는 0001부터 시작한다. 예를 들어 students.txt에 S0001, S0002, S0005가 존재한다면 다음 학생 ID는 S0006으로 부여한다.

새로 부여할 번호가 9999를 초과하는 경우에는 새로운 데이터를 등록할 수 없으며 오류로 처리한다. 삭제된 레코드의 내부 식별자는 현재 데이터 파일에 더 이상 존재하지 않으므로, 이후 ID 부여 과정에서 다시 사용될 수 있다. 단, 현재 존재하는 다른 레코드와 동일한 내부 식별자를 동시에 사용할 수는 없다.

5.1.14 문법 오류와 의미 오류

각 필드는 먼저 해당 데이터 요소의 문법 형식을 만족해야 한다.

모든 필드가 문법적으로 올바른 경우에만 데이터 간의 중복, 참조 관계, 날짜 및 일정 관계와 같은 의미 규칙을 검사한다.

문법적으로 올바르지 않은 파일을 참조하여 이후의 관계 검사를 수행하지 않는다. 구체적인 오류 코드와 프로그램의 처리 방법은 오류 처리 절에서 정의한다.

5.2 파일 형식

5.2.1 사용자 계정 파일

사용자 계정 파일은 프로그램에 등록된 사용자 계정의 사용자 ID, 비밀번호, 사용자 역할을 저장한다.
파일명은 `users.txt`이다.

하나의 사용자 계정은 하나의 레코드로 저장한다.

헤더:

사용자ID<TAB>비밀번호<TAB>사용자역할

문법 형식:

〈사용자 ID〉 <TAB> 〈비밀번호〉 <TAB> 〈사용자 역할〉

〈사용자 ID〉는 4.1.1절의 사용자 ID, 〈비밀번호〉는 4.3절의 비밀번호, 〈사용자 역할〉은 4.4절의 사용자 역할 문법 형식을 각각 만족해야 한다.

비밀번호는 별도의 암호화 또는 해시 변환을 하지 않고 사용자가 설정한 문자열 그대로 `users.txt`에 저장한다. 로그인 시 사용자가 입력한 비밀번호와 `users.txt`에 저장된 비밀번호를 4.3절의 동치 비교 규칙에 따라 비교한다.

예를 들어 다음은 올바른 사용자 계정 레코드이다.

std01<TAB>academy01<TAB>학생

teacher1<TAB>Class123!<TAB>강사

반면 다음은 올바른 사용자 계정 레코드가 아니다.

std01<TAB>academy01

std01<TAB>academy01<TAB>학생<TAB>추가

첫 번째 레코드는 사용자 역할 필드가 없고, 두 번째 레코드는 필드가 하나 많다.

의미 규칙:

- 서로 다른 두 사용자 계정은 서로 동치인 사용자 ID를 가질 수 없다.
- 역할이 "학생"인 사용자 계정은 정확히 하나의 학생 레코드와 연결되어야 한다.
- 역할이 "강사"인 사용자 계정은 정확히 하나의 강사 레코드와 연결되어야 한다.
- 역할이 "원장"인 사용자 계정은 학생 또는 강사 레코드와 연결되지 않는다.
- 비밀번호는 평문으로 저장하며, 저장된 문자열을 임의로 변환하지 않는다.
- users.txt에는 역할이 원장인 계정이 정확히 하나 존재해야 한다. 원장 계정이 없거나 둘 이상 존재하면 의미 규칙 오류로 처리하고 프로그램 시작을 차단한다.

학생 및 강사와의 연결은 `users.txt`에 별도의 연결 ID를 중복 저장하지 않고 `students.txt`와 `teachers.txt`에서 사용자 ID를 참조하는 방식으로 처리한다.

동치 비교: 두 사용자 계정의 사용자 ID가 서로 동치인 경우 두 레코드는 같은 사용자 계정을 나타내는 것으로 간주한다.

로그인 시 입력한 비밀번호와 저장된 비밀번호의 문자열 전체가 완전히 일치하는 경우에만 비밀번호가 일치하는 것으로 간주한다.

의미 규칙:

- 서로 다른 두 사용자 계정은 서로 동치인 사용자 ID를 가질 수 없다.
- 역할이 "학생"인 사용자 계정은 정확히 하나의 학생 레코드와 연결되어야 한다.
- 역할이 "강사"인 사용자 계정은 정확히 하나의 강사 레코드와 연결되어야 한다.
- 역할이 "원장"인 사용자 계정은 학생 또는 강사 레코드와 연결되지 않는다.

학생 및 강사와의 연결은 `users.txt`에 별도의 연결 ID를 중복 저장하지 않고 `students.txt`와 `teachers.txt`에서 사용자 ID를 참조하는 방식으로 처리한다.

동치 비교: 두 사용자 계정의 사용자 ID가 서로 동치인 경우 두 레코드는 같은 사용자 계정을 나타내는 것으로 간주한다.

5.2.2 학생 파일

학생 파일은 학생 ID, 이름, 연락처, 사용자 ID, 학생 상태를 저장한다. 파일명은 `students.txt`이다.

하나의 학생은 하나의 레코드로 저장한다.

헤더:

학생ID<TAB>이름<TAB>연락처<TAB>사용자ID<TAB>학생상태

문법 형식:

〈학생 ID〉 <TAB> 〈사람 이름〉 <TAB> 〈연락처〉 <TAB> 〈사용자 ID〉 <TAB> 〈학생 상태〉

각 필드는 각각 4장에서 정의한 해당 데이터 요소의 문법 형식을 만족해야 한다.

예를 들어 다음은 올바른 학생 레코드이다.

S0001<TAB>김민수<TAB>010-1234-5678<TAB>std01<TAB>재원

의미 규칙:

- 서로 다른 두 학생 레코드는 같은 학생 ID를 가질 수 없다.
- 사용자 ID는 `users.txt`에 실제로 존재해야 한다.
- 해당 사용자 계정의 역할은 "학생"이어야 한다.
- 서로 다른 두 학생이 같은 사용자 ID를 참조할 수 없다.
- 학생 이름과 연락처는 다른 학생과 중복될 수 있다.

동치 비교: 두 학생 레코드의 학생 ID가 서로 동치인 경우 같은 학생을 나타내는 것으로 간주한다.

학생의 퇴원 여부는 학생 상태를 통해 나타내며, 퇴원한다고 해서 학생 레코드와 기존 수강 기록을 즉시 삭제하지 않는다.

5.2.3 강사 파일

강사 파일은 강사 ID, 이름, 연락처, 사용자 ID, 강사 상태를 저장한다. 파일명은 `teachers.txt`이다.

강사가 담당하는 과목이나 반의 목록을 이 파일에 중복하여 저장하지 않는다. 담당 반과 과목은 `classes.txt`에서 해당 강사 ID를 참조하는 반을 찾아 확인한다.

헤더:

강사ID<TAB>이름<TAB>연락처<TAB>사용자ID<TAB>강사상태

문법 형식:

<강사 ID> <TAB> <사람 이름> <TAB> <연락처> <TAB> <사용자 ID> <TAB> <강사 상태>

예를 들어 다음은 올바른 강사 레코드이다.

T0001<TAB>이준호<TAB>010-2345-6789<TAB>teacher1<TAB>재직

의미 규칙:

- 서로 다른 두 강사 레코드는 같은 강사 ID를 가질 수 없다.
- 사용자 ID는 `users.txt`에 실제로 존재해야 한다.
- 해당 사용자 계정의 역할은 "강사"이어야 한다.
- 서로 다른 두 강사가 같은 사용자 ID를 참조할 수 없다.
- 강사의 이름과 연락처는 서로 다른 강사 사이에서 중복될 수 있다.

동치 비교: 두 강사 레코드의 강사 ID가 서로 동치인 경우 같은 강사를 나타내는 것으로 간주한다.

강사가 퇴직하더라도 해당 강사 레코드와 기존 담당 기록은 삭제하지 않는다.

5.2.4 과목 파일

과목 파일은 과목 ID와 과목명을 저장한다. 파일명은 `subjects.txt`이다.

과목은 실제로 개설되는 반과 구별한다. 하나의 과목으로 서로 다른 강사, 일정, 정원을 가진 여러 반을 개설할 수 있다.

헤더:

과목ID<TAB>과목명

문법 형식:

〈과목 ID〉 <TAB> 〈과목명〉

예를 들어 다음은 올바른 과목 레코드이다.

P0001<TAB>수학

P0002<TAB>중등 수학

의미 규칙:

- 서로 다른 두 과목 레코드는 같은 과목 ID를 가질 수 없다.
- 과목명은 서로 다른 과목 사이에서 중복될 수 있다.

따라서 다음 두 레코드는 동시에 존재할 수 있다.

P0001<TAB>수학

P0002<TAB>수학

두 과목은 과목명이 같지만 과목 ID가 다르므로 서로 다른 과목이다.

동치 비교: 두 과목 레코드의 과목 ID가 서로 동치인 경우 같은 과목으로 간주한다.

5.2.5 반 파일

반 파일은 실제로 개설된 반의 반 ID, 과목 ID, 반 이름, 담당 강사 ID, 요일 목록, 시작 교시, 종료 교시, 시작 날짜, 종료 날짜, 정원, 반 상태를 저장한다. 파일명은 `classes.txt`이다.

현재 수강 인원은 별도의 필드로 저장하지 않는다. 해당 반에 연결된 "수강중" 상태의 수강 등록 수를 이용하여 계산한다.

헤더:

반ID<TAB>과목ID<TAB>반이름<TAB>강사ID<TAB>요일목록<TAB>시작교시<TAB>종료교시<TAB>시작날짜<TAB>종료날짜<TAB>정원<TAB>반상태

문법 형식:

<반 ID> <TAB> <과목 ID> <TAB> <반 이름> <TAB> <강사 ID> <TAB> <요일 목록> <TAB> <시작 교시> <TAB> <종료 교시> <TAB> <시작 날짜> <TAB> <종료 날짜> <TAB> <정원> <TAB> <반 상태>

예를 들어 다음은 올바른 반 레코드이다.

C0001<TAB>P0001<TAB>A반<TAB>T0001<TAB>월
수<TAB>3<TAB>4<TAB>2026-09-02<TAB>2026-12-16<TAB>10<TAB>개설

의미 규칙:

- 서로 다른 두 반은 같은 반 ID를 가질 수 없다.
- 과목 ID는 `subjects.txt`에 실제로 존재해야 한다.
- 강사 ID는 `teachers.txt`에 실제로 존재해야 한다.
- 새로 개설하거나 수정하는 반의 담당 강사는 재직 상태여야 한다. 이미 저장된 반의 담당 강사가 이후 퇴직 상태로 바뀌는 것은 허용하며, 무결성 오류로 보지 않는다.
- 시작 날짜는 종료 날짜보다 이전이거나 같아야 한다.
- 시작 교시는 종료 교시보다 이전이거나 같아야 한다.
- 요일 목록은 4.11.1절의 의미 규칙을 만족해야 한다.
- 정원은 1 이상의 정수여야 한다.

- 동일한 과목에 속하는 “개설” 상태인 서로 다른 두 반은 같은 반 이름을 가질 수 없다. 서로 다른 과목에 속하는 반끼리는 같은 반 이름을 가질 수 있다.

반 상태

파일에 저장하는 반 상태는 다음 두 값 중 하나이다.

개설, 폐강

"개설"은 정상적으로 운영되는 반을 의미하며, "폐강"은 종료 날짜와 관계없이 더 이상 운영하지 않는 반을 의미한다.

5.2.6 수강 등록 파일

수강 등록 파일은 학생과 반 사이의 수강 등록 관계를 저장한다. 파일명은 `enrollments.txt`이다.

헤더:

등록ID<TAB>학생ID<TAB>반ID<TAB>등록일시<TAB>등록상태<TAB>취소일시

문법 형식:

〈수강 등록 ID〉 <TAB> 〈학생 ID〉 <TAB> 〈반 ID〉 <TAB> 〈등록 일시〉 <TAB> 〈수강 등록 상태〉 <TAB> 〈취소 일시〉

수강 등록 상태는 다음 두 값 중 하나이다.

수강중, 취소

"수강중"은 학생과 반 사이의 수강 등록 관계가 취소되지 않고 유효한 상태를 의미한다. "취소"는 해당 수강 등록이 취소된 상태를 의미한다.

상태가 "수강중"인 경우 취소 일시는 존재하지 않으므로 "-"를 저장한다.

예를 들어 다음은 올바른 수강 등록 레코드이다.

E0001<TAB>S0001<TAB>C0001<TAB>2026-08-27 15:30<TAB>수강중<TAB>-

취소가 발생한 경우에는 다음과 같이 저장한다.

E0002<TAB>S0002<TAB>C0001<TAB>2026-08-28 14:00<TAB>취소<TAB>2026-09-05 16:20

의미 규칙:

- 서로 다른 두 수강 등록은 같은 수강 등록 ID를 가질 수 없다.
- 학생 ID는 `students.txt`에 실제로 존재해야 한다.
- 반 ID는 `classes.txt`에 실제로 존재해야 한다.
- 상태가 "수강중"이면 취소 일시는 반드시 "-"이어야 한다.
- 상태가 "취소"이면 취소 일시는 4.9절의 올바른 일시여야 한다.
- 취소 일시는 등록 일시보다 빠를 수 없다.
- 동일한 학생과 동일한 반에 대해 "수강중" 상태의 수강 등록이 둘 이상 존재할 수 없다.

정원 계산 시에는 해당 반의 "수강중" 상태 수강 등록만 계산한다.

정원 초과, 학생 수업 시간 충돌, 신청 가능 기간 등은 파일 자체의 문자열 문법이 아니라 수강 신청 기능의 의미 규칙에서 검사한다.

동치 비교: 두 수강 등록의 수강 등록 ID가 서로 동치인 경우 같은 수강 등록을 나타내는 것으로 간주한다.

5.2.7 가상 현재 일시 파일

가상 현재 일시 파일은 프로그램 전체에서 현재 시각의 기준으로 사용하는 가상 현재 일시와 그 변경 이력을 저장한다. 파일명은 `datetime.txt`이다.

`datetime.txt`는 다른 데이터 파일과 달리 헤더를 사용하지 않으며, 한 줄에 하나씩 일시를 저장한다. 마지막 줄이 현재 유효한 가상 현재 일시이고, 그 이전 줄들은 과거에 지정되었던 가상 현재 일시의 이력이다.

문법 형식:

〈일시〉

예를 들어 다음은 올바른 파일의 내용이다.

2026-09-27 14:30

2026-09-28 09:00

2026-10-15 09:00

이 경우 2026-10-15 09:00이 현재 유효한 가상 현재 일시이며, 앞의 두 줄은 이력이다. 각 줄의 값은 4.9절에서 정의한 일시의 문법 및 의미 규칙을 만족해야 한다. `datetime.txt`는 빈 파일일 수 없으며, 최소 한 줄의 일시를 저장한다. 줄들은 오래된 것부터 최근 것 순으로 정렬되어 있어야 한다. 파일의 마지막에는 개행이 있어도 되고 없어도 된다. 가상 현재 일시를 과거로 변경할 수 있는지, 변경 권한이 누구에게 있는지 등의 규칙은 4.14절에서 정의한다.

5.3 부가 확인 항목

항목	처리
필드 값 양끝 공백	콘솔에서 문자열을 입력받는 경우, 별도로 명시하지 않는 한 입력값의 앞과 뒤에 있는 표준공백을 제거한 뒤 해당 데이터 요소의 문법 및 의미 규칙을 검사한다. 단, 비밀번호는 앞뒤 표준공백을 제거하지 않고 입력값 그대로 검사한다. 데이터 파일에서 읽은 값은 콘솔 입력과 달리 앞뒤 표준공백을 자동으로 제거하지 않는다. 따라서 해당 데이터 요소에서 허용되지 않는 앞뒤 표준공백이 포함된 경우 형식 오류(E_FILE_FORMAT)로 처리한다.
과목명 중복	과목ID만 유일하면 되며, 과목명 자체의 중복은 허용한다(경고 없음).
파일 내 정렬 순서	ID 순서를 요구하지 않는다. 프로그램은 순서에 상관없이 전체를 읽어 처리한다.
폐강·퇴원 등 과거 기록	폐강된 반과 퇴원한 학생은 신규 수강 신청 대상에서 제외한다. 현재 운영·수강 대상만 표시하는 조회에서는 해당 기록을 제외한다. 원장의

	전체 목록·상세 조회 및 과거 이력 조회에서는 폐강·퇴원 기록도 포함하며 상태를 함께 표시한다
빈 선택 파일 등 극단 사례	파일에 데이터 줄이 하나도 없는 경우(헤더만 있음)는 오류가 아니라 "해당 항목 없음"으로 처리한다.

5.4 무결성 확인 및 처리

데이터 파일 사이에는 다음과 같은 참조 관계가 존재한다.

- `students.txt`의 사용자 ID → `users.txt`
- `teachers.txt`의 사용자 ID → `users.txt`
- `classes.txt`의 과목 ID → `subjects.txt`
- `classes.txt`의 강사 ID → `teachers.txt`
- `enrollments.txt`의 학생 ID → `students.txt`
- `enrollments.txt`의 반 ID → `classes.txt`

다른 파일의 식별자를 참조하는 필드는 해당 식별자를 가진 데이터가 실제로 존재해야 한다. 존재하지 않는 식별자를 참조하는 레코드는 의미상 올바르지 않다. 또한 각 파일에서 해당 데이터를 식별하기 위한 고유 ID는 중복될 수 없다.

파일 검사는 다음 여덟 단계의 순서로 수행하는 것을 원칙으로 한다. ①~④는 파일 자체의 형식 검사, ⑤~⑥은 식별자 검사, ⑦~⑧은 데이터 간 의미 규칙 검사이다.

- 파일 존재 여부와 읽기 가능 여부를 확인한다.
- 파일의 인코딩을 확인한다.
- 각 행의 필드 수와 공백·빈 줄 등 행 단위 문법을 확인한다.
- 각 필드의 형식과 값의 범위를 확인한다.
- 각 파일 내부의 ID 중복 여부를 확인한다.

- 다른 파일을 참조하는 ID가 실제로 존재하는지 확인한다.
- 일정 충돌, 정원 초과, 수강 중복 등 데이터 간의 의미 규칙을 확인한다.
- 가상 현재 일시가 "기록 일시"보다 이전인지 확인한다.

앞 단계에서 파일 자체의 문법 오류가 발견된 경우, 해당 파일을 이용해야 하는 이후의 참조 관계·의미 규칙 검사는 수행하지 않는다.

아래 표는 위 여덟 단계를, 이 프로그램이 실제로 사용하는 파일과 오류 코드에 맞추어 정리한 것이다.

검사 단계	정한 대상	오류 코드	처리
① 파일 존재·읽기 가능	7개 데이터 파일(users.txt, students.txt, teachers.txt, subjects.txt, classes.txt, enrollments.txt, datetime.txt)	E_FILE_MISSING, E_FILE_READ	즉시 종료
② UTF-8 디코딩	모든 파일	E_FILE_FORMAT	즉시 종료
③ 행의 필드 수·문법	5.1의 필드 수, 빈 줄·주석 금지, 공백 규칙	E_FILE_FORMAT	즉시 종료
④ 필드별 형식·범위	5.2의 ID 형식, 날짜·시각·정원·상태 값 열거형	E_FILE_FORMAT	즉시 종료
⑤ ID 중복	각 파일 1번 필드	E_DUP_ID	즉시 종료
⑥ 참조 존재 여부	5.2의 참조 무결성 목록(위 관계 참고)	E_REF_MISSING	즉시 종료

⑦ 수업 시간·정원·등록 관계	classes.txt의 강사 시간 충돌, enrollments.txt의 정원·학생 시간 충돌	E_TEACHER_CONF LICT, E_STUDENT_CONF LICT, E_CLASS_FULL	즉시 종료
⑧ 가상 시각과 상태	datetime.txt의 마지막 줄(현재 가상 현재 일시)이 4.14절에서 정의한 "기록 일시" 집합(① enrollments.txt의 수강 등록 일시, ② enrollments.txt의 수강 취소 일시, ③ datetime.txt에 저장된 과거 가상 현재 일시 이력)의 최댓값보다 이전인지 여부. classes.txt의 수업 시작일·종료일 등 미래 예정 시점은 비교 대상에서 제외한다	E_TIME_BACKWAR D	즉시 종료

본 프로그램은 데이터 파일의 무결성이 하나라도 깨진 상태에서는 어떤 기능도 수행하지 않는다.

①~⑧ 중 어느 단계에서든 오류가 발견되면 해당 오류 코드와 원인이 되는 파일·행 번호를 출력하고 프로그램을 즉시 종료한다. 자동 수정은 5.1.12절에 따라 수행하지 않으며, 경고 후 계속 진행하는 처리도 없다. 사용자가 데이터 파일을 올바른 상태로 복구한 뒤 다시 실행해야 한다.

⑧단계 검사의 판정 기준이 되는 "기록 일시"의 정확한 범위는 4.14절 "의미 규칙" 항목에 정의되어 있으며, 본 절에서는 그 기준을 그대로 사용한다. 실행 중 가상 현재 일시를 변경할 때에는 4.14절의 역행 판정 기준 일시를 적용한다. 요약하면 다음과 같다.

포함: 수강 등록 일시, 수강 취소 일시, 과거 가상 현재 일시 이력

제외: 반 수업 시작일·종료일·수업 시간 등 아직 발생하지 않은 미래 예정 시점,

사용자·학생·강사·과목의 등록 시각 등 참조용 메타데이터

예를 들어 datetime.txt의 마지막 줄이 2026-10-10 09:00이고, enrollments.txt에 등록 일시 2026-10-03 13:00인 레코드가 존재하며, classes.txt에 수업 시작일 2026-12-01인 반이 등록되어 있다고 하자. 이때 수업 시작일 2026-12-01은 비교 대상이 아니므로, 하한선은 2026-10-03 13:00이고 현재 값 2026-10-10 09:00은 옳바르다. 반면 사용자가 datetime.txt의 마지막 줄을 2026-10-01 09:00으로 직접 수정하면 등록 일시 2026-10-03 13:00보다 이전이므로 E_TIME_BACKWARD 오류로 처리한다.

또한 datetime.txt의 이력 줄 자체가 손상된 경우(형식 위반·비정렬·마지막 줄이 이력보다 과거) 역시 E_TIME_BACKWARD 오류로 처리한다. 세부 판정 기준은 4.14절 "저장" 항목에 정의되어 있다.

여러 파일을 함께 변경하는 작업의 저장 절차

변경 대상 파일 전부의 새 내용을 각각 임시 파일에 먼저 기록하고, 모든 기록에 성공한 경우에만 교체를 시작한다. 하나라도 실패하면 임시 파일을 모두 삭제하고 작업 전체를 실패로 처리하며, 이때 원본은 변경되지 않는다.

교체 도중에 프로그램이 비정상 종료되어 일부 파일만 교체된 경우는 다음 실행의 무결성 검사에서 참조 오류로 탐지한다.

6. 주 프롬프트

6.1 메인 메뉴

6.1.1 역할별 메인메뉴 화면

로그인에 성공하면 역할에 따라 아래 형식으로 메뉴를 출력하고, 번호 입력을 받는다. 학생과 강사의 메뉴 제목 줄에는 로그인한 사용자의 이름을 함께 표시한다. 원장은 계정에 이름을 저장하지 않으므로 표시하지 않는다.

===== 원장 메뉴 =====

1. 학생 관리
 2. 강사 관리
 3. 과목 관리
 4. 반 관리
 5. 수강 등록 조회·취소
 6. 가상 현재 일시 변경
 7. 무결성 검사
 8. 비밀번호 관리
 9. 로그아웃
 0. 프로그램 종료
- 번호를 입력하세요: ⌋

===== 강사 메뉴 ({{이름}}님) =====

1. 담당 반 조회
 2. 담당 반 수강생 조회
 3. 수업 일정 조회
 9. 로그아웃
 0. 프로그램 종료
- 번호를 입력하세요: ⌋

===== 학생 메뉴 ({이름}님) =====

1. 내 정보 조회
 2. 수강 가능한 반 조회
 3. 반 상세 조회
 4. 수강 신청
 5. 내 수강 목록 조회
 6. 수강 취소
 9. 로그아웃
 0. 프로그램 종료
- 번호를 입력하세요: ▯

하위 메뉴(학생·강사·과목·반 관리)도 같은 형식을 사용하며, 제목 줄만 "===== {상위 메뉴명} > {하위 메뉴명} ====="로 바꾼다.

===== 학생 관리 =====

1. 목록 조회
 2. 상세 조회
 3. 등록
 4. 수정
 5. 삭제
 0. 이전 메뉴
- 번호를 입력하세요: ▯

6.1.2 메뉴 입력 오류

규칙(번호 하나만 허용, 01~+1~1.0~1,2~빈 입력 불허)에 대해 다음 메시지를 공통으로 사용한다.

[E_MENU_CHOICE] 잘못된 입력입니다. 화면에 표시된 번호 중 하나만 입력하세요.

오류 메시지 출력 후에는 같은 메뉴를 다시 출력하고 입력을 재요청한다. 별도의 "다시 시도하시겠습니까?" 확인은 받지 않는다.

6.1.3 확인(Y/N) 프롬프트

삭제, 수강 취소, 가상 시각 변경, 프로그램 종료처럼 결과가 중요한 작업은 실행 직전 다음 형식으로 확인을 받는다. Enter만 입력하면 N으로 처리한다.

{작업 설명} 하시겠습니까? (Y/N, 기본값 N): ▯

예시는 다음과 같다 :

반 C0001(수학 A반)을 폐강하시겠습니까? (Y/N, 기본값 N):

Y, y 이외의 모든 입력(빈 입력 포함)은 N으로 처리하고 다음 메시지와 함께 이전 메뉴로 복귀한다.

작업을 취소했습니다. 변경된 내용이 없습니다.

6.1.4 프로그램 종료

최상위 메뉴에서 0을 입력하면 종료 여부를 확인한다.

프로그램을 종료하시겠습니까? (Y/N, 기본값 N):

Y 입력 시는 다음과 같다 :

프로그램을 종료합니다.

6.1.5 빈 입력 및 입력 중 취소

입력 상황	빈 입력(Enter 만 입력) 처리
메뉴 번호 입력	잘못된 입력으로 처리하고 같은 메뉴에서 재입력한다.
로그인 등 필수값 입력	[E_INPUT_EMPTY] 필수 값을 입력하세요.를 출력하고 같은 항목을 재입력한다.
상세 조회 등 대상을 지정하기 위한 ID 입력	해당 작업을 취소하고 이전 메뉴로 돌아간다.
Y/N 확인 프롬프트	N으로 처리하여 작업을 취소한다.
기존 값을 변경하는 입력에서 별도로 기존 값 유지가 명시된 경우	기존 값을 그대로 유지한다.
그 밖의 입력	해당 기능에서 별도로 정의한 규칙을 따른다.

학생·강사 등록, 반 개설처럼 여러 필드를 순서대로 입력하는 작업에서는 입력 도중 사용자가 작업을 취소할 수 있도록 각 부 프롬프트에서 “/cancel”을 입력할 수 있다. “/cancel”을 입력하면 그때까지 입력한 값은 모두 폐기하고 데이터 파일을 변경하지 않은 채 해당 작업의 상위 메뉴로 돌아간다. 여러 필드 입력 작업에서 필수 항목에 빈 입력을 한 경우에는 작업을 취소하지 않고 “[E_INPUT_EMPTY] 필수 값을 입력하세요.”를 출력한 뒤 같은 항목을 다시 입력받는다.

6.1.6 조회 결과 정렬

조회 결과에 별도의 정렬 기준이 명시되어 있지 않은 경우 해당 데이터의 내부 식별자를 기준으로 오름차순 정렬한다.

둘 이상의 정렬 기준이 있는 경우 앞에 명시된 기준을 우선하며, 모든 선행 기준이 같은 경우 해당 데이터의 내부 식별자를 오름차순으로 비교한다.

요일을 기준으로 정렬하는 경우 월 → 화 → 수 → 목 → 금 → 토 → 일 순서를 사용한다.

반의 조회상 운영 상태는 가상 현재 일시의 날짜를 기준으로 다음과 같이 판단한다.

- 반 상태가 “폐강”이면 폐강
- 반 상태가 “개설”이고 가상 현재 날짜가 시작 날짜보다 이전이면 모집중
- 반 상태가 “개설”이고 시작 날짜 ≤ 가상 현재 날짜 ≤ 종료 날짜이면 진행중
- 반 상태가 “개설”이고 가상 현재 날짜가 종료 날짜보다 이후이면 종료

“모집중”, “진행중”, “종료”는 조회를 위한 표시상 구분이며 데이터 파일에 저장되는 반 상태는 아니다.

6.2 로그인

6.2.1 로그인 화면

프로그램 시작 시(시작 무결성 검사 통과 후) 다음 화면을 출력한다.

```
===== 학원 운영 관리 프로그램 =====
사용자 ID: 
비밀번호: (입력 시 화면에 표시되지 않음, getpass 사용)
```

사용자 ID, 비밀번호는 입력 규칙에 따라 빈 값을 허용하지 않는다. Enter만 입력하면 다음 메시지를 출력하고 같은 항목을 다시 입력받는다.

```
[E_INPUT_EMPTY] 필수 값을 입력하세요.
```

사용자 ID에 탭이나 줄바꿈이 포함된 경우(붙여넣기 등으로 발생 가능)도 동일하게 E_INPUT_EMPTY가 아닌 형식 오류로 처리한다.

[E_FILE_FORMAT] 사용자 ID 형식이 올바르지 않습니다.

사용자 ID 입력은 종료 명령 확인 → 빈 입력 확인 → 사용자 ID 문법 형식 검사 → 계정 존재 여부 확인 순서로 처리한다. 사용자 ID 입력값이 0이면 이후의 사용자 ID 형식 검사를 수행하지 않고 6.7.1절의 프로그램 종료 절차로 이동한다.

6.2.2 인증 실패

"ID 없음"과 "비밀번호 불일치"는 같은 메시지를 사용한다.

[E_AUTH_FAIL] 로그인 정보가 올바르지 않습니다.

인증 실패 후에는 사용자 ID 입력부터 다시 받는다(기밀성 보장을 위해 아이디와 비밀번호 중 어느 쪽이 틀렸는지 알려주지 않기 위해 비밀번호만 다시 받지 않는다)

비활성 계정:

사용할 수 없는 계정입니다.

사용자 ID와 비밀번호가 일치하더라도 연결된 학생 상태가 퇴원이거나 강사 상태가 퇴직이면 비활성 계정으로 판단하고 로그인을 거부한다. "사용할 수 없는 계정입니다."를 출력하고 사용자 ID 입력 단계로 돌아간다. 비활성 계정의 로그인 거부도 인증 실패 횟수에 포함한다.

계정과 학생·강사의 연결 자체가 잘못된 경우에는 비활성 계정으로 처리하지 않고 데이터 무결성 오류로 처리한다.

6.2.3 연속 실패 처리

로그인 실패 횟수는 사용자 ID와 비밀번호의 입력이 완료되어 실제 로그인 가능 여부를 검사한 시도 중, 로그인에 성공하지 못한 횟수를 의미한다.

다음 경우에는 로그인 실패 횟수를 1 증가시킨다.

- 입력한 사용자 ID에 해당하는 계정이 존재하지 않는 경우
- 사용자 ID는 존재하지만 비밀번호가 일치하지 않는 경우
- 사용자 ID와 비밀번호가 일치하지만 해당 계정을 사용할 수 없는 경우

빈 입력 또는 사용자 ID·비밀번호의 문법 형식 오류는 로그인 정보의 검증 단계까지 진행되지 않은 입력 오류이므로 로그인 실패 횟수에 포함하지 않는다. 이러한 입력 오류가 발생하더라도 기존 로그인 실패 횟수는 초기화하지 않는다.

로그인에 성공하면 로그인 실패 횟수를 0으로 초기화한다.

로그인 실패 횟수가 연속 5회가 되면 다음 메시지를 출력한 뒤 별도의 확인 없이 프로그램을 종료한다.

[E_AUTH_FAIL] 로그인 정보가 올바르지 않습니다.
로그인 시도 횟수를 초과했습니다. 프로그램을 종료합니다.

이후 별도의 확인 없이 프로그램을 종료한다(데이터 변경은 따로 제공하지 않는다)

6.2.4 로그인 성공

로그인 성공 메시지는 역할별로 다음과 같다.

학생·강사:

로그인되었습니다. 안녕하세요, {이름}님.

원장:

로그인되었습니다. 안녕하세요, 원장님.

이후 6.1절의 역할별 메인메뉴를 출력한다. 실패 횟수는 0으로 초기화한다(같은 실행 중 이후 로그아웃 후 재로그인 시에도 실패 횟수는 이어서 누적하지 않고 새로 센다.)

6.2.5 로그아웃

역할별 메뉴에서 9를 입력하면 확인 없이 즉시 로그아웃하고 6.2.1절의 로그인 화면으로 돌아간다.

로그아웃되었습니다.

===== 학원 운영 관리 프로그램 =====
사용자 ID: ▯

6.2.6 로그인 보안 범위

본 시스템의 로그인은 프로그램 내부에서 사용자 계정 및 역할에 따른 기능 접근을 구분하는 것을 목적으로 한다. 계정 정보와 비밀번호는 데이터 파일에 저장하며, 프로젝트 설계에 따라 비밀번호는 평문으로 저장한다. 저장 파일에 직접 접근할 수 있는 사용자는 계정 또는 역할 정보를 변경할 수 있으므로, 외부 파일 접근까지 보호하는 보안 기능은 본 프로젝트의 구현 범위에 포함하지 않는다.

6.3 가상 현재 일시

원장 메뉴 6번 기능. 일시 문법(YYYY-MM-DD HH:MM)과 잠정 규칙(같은 시각 허용, 과거로 되돌릴 수 없음)을 그대로 사용한다.

6.3.1 현재 값 표시 및 입력

현재 가상 일시: {현재가상일시}
새 가상 일시를 입력하세요 (형식: YYYY-MM-DD HH:MM):

Enter만 입력한 경우, "수업 날짜·시간·정원은 묵시적 기본값 없음" 원칙에 따라 기본값을 적용하지 않고 재입력을 요구한다.

[E_INPUT_EMPTY] 필수 값을 입력하세요.

6.3.2 형식·의미 오류

정의된 문법(존재하지 않는 날짜, 24:00 같은 잘못된 시각 등)을 만족하지 못하면 다음 메시지를 출력하고 같은 항목을 다시 입력받는다.

[E_FILE_FORMAT] 일시 형식이 올바르지 않습니다. 예: 2026-09-27 15:30

6.3.3 시각 역행

새 가상 현재 일시가 4.14절의 역행 판정 기준 일시보다 이전이면 변경을 거부한다. 기준 일시와 동일한 값은 허용한다.

[E_TIME_BACKWARD] 새 가상 일시는 역행 판정 기준 일시({기준일시})보다 이전일 수 없습니다.

6.3.4 변경 확인

형식·역행 검사를 모두 통과하면 6.1.3절의 공통 확인 프롬프트를 사용한다.

가상 현재 일시를 {현재가상일시} → {새가상일시}(으)로 변경하시겠습니까? (Y/N, 기본값 N): ▮

N(또는 Enter) 입력 시는 다음과 같이 처리한다 :

작업을 취소했습니다. 가상 현재 일시가 변경되지 않았습니다.

6.3.5 저장 결과

저장 직전 재검사를 통과하면 **datetime.txt** 끝에 새 일시를 한 줄 추가한 뒤 다음을 출력한다.

[OK_TIME] 가상 현재 일시가 {새가상일시}(으)로 변경되었습니다.

저장 직전 재검사에서 외부 수정이 감지된 경우:

[E_CONCURRENT_EDIT] 데이터 파일이 변경되었습니다. 다시 조회한 뒤 시도하세요.

저장 자체(임시 파일 생성·교체)가 실패한 경우:

[E_SAVE] 저장하지 못했습니다. 가상 현재 일시는 변경되지 않았습니다.

두 경우 모두 원장 메뉴로 복귀하며 **datetime.txt**는 변경되지 않는다.

6.4 학생 메뉴

학생은 다음 기능을 사용할 수 있다.

===== 학생 메뉴 ({이름}님) =====

1. 내 정보 조회
 2. 수강 가능한 수업 조회
 3. 반 상세 조회
 4. 수강 신청
 5. 내 수강 목록 조회
 6. 수강 취소
 9. 로그아웃
 0. 프로그램 종료
- 번호를 입력하세요:

6.4.1 내 정보 조회

===== 학생 메뉴 ({이름}님) > 내 정보 조회 =====

학생 ID: {학생ID}
이름: {학생이름}

0. 이전 메뉴
번호를 입력하세요:

6.4.2 수강 가능한 반 조회

학생은 현재 신청할 수 있는 반의 목록을 조회할 수 있다.

로그인한 학생이 현재 신청 조건을 모두 만족하는 반만 조회한다. 반의 개설 상태, 신청 가능 기간, 학생의 재원 여부, 정원, 중복 등록 및 학생 일정 충돌 여부를 기준으로 판단한다.

수강 가능한 반 목록에는 로그인한 학생이 재원 상태이고, 반이 개설 상태이며, 가상 현재 날짜가 반의 시작 날짜 이전인 반만 포함한다. 해당 반은 현재 인원이 정원보다 적어야 하고, 학생의 동일 반 수강중 등록이 없어야 하며, 기존 수강중인 개설 상태 반과 일정이 충돌하지 않아야 한다.

모든 조건을 만족하는 반이 없으면 “현재 수강 신청할 수 있는 반이 없습니다.”를 출력한다. 목록에 표시된 반을 선택하더라도 실제 신청 시 모든 조건을 다시 검사한다.

===== 학생 메뉴 ({이름}님) > 수강 가능한 반 조회 =====

반 ID | 반 이름 | 과목명 | 담당 강사 | 수업 요일 | 교시 | 수업 기간 | 현재 인원/정원

C0001 | A반 | 수학 | 이준호 | 월/수 | 3교시 | 2026-09-01~2026-12-20 | 8/10

C0002 | B반 | 영어 | 이강사 | 화/목 | 2교시 | 2026-09-01~2026-12-20 | 6/10

상세 조회할 반 ID를 입력하세요.

0. 이전 메뉴

입력:

6.4.3 반 상세 정보 조회

특정 반을 선택하면 해당 반의 상세 정보를 확인할 수 있다.

===== 학생 메뉴 ({이름}님) > 반 상세 정보 조회 =====

반 ID: {반ID}

반 이름: {반이름}

과목명: {과목명}

담당 강사: {강사이름}

수업 요일: {요일}

교시: {교시}

수업 기간: {수업기간}

현재 수강 인원: {현재인원}

정원: {정원}

상태: {반상태}

0. 이전 메뉴

번호를 입력하세요:

6.4.4 수강 신청

===== 학생 메뉴 ({이름}님) > 수업 신청 =====

수강 신청할 반 ID를 입력하세요:

반 아이디가 확인 되면 수강 신청 가능 여부를 확인한다.

===== 학생 메뉴 ({이름}님) > 수업 신청 > {반ID} =====

반 ID: {반ID}
반 이름: {반이름}
과목명: {과목명}
담당 강사: {강사이름}
업 요일: {요일}
교시: {교시}
현재 수강 인원: {현재인원} / {정원}

수강 신청하시겠습니까? (Y/N, 기본값 N):

Y를 입력하고 수강 신청 조건을 모두 만족하면 다음과 같이 출력한다.

수강 신청이 완료되었습니다.

수강 신청 조건을 만족하지 못하는 경우 해당 사유를 안내한다.

정원이 가득 찬 경우:

[E_CLASS_FULL] 해당 반의 정원이 가득 찼습니다. 수강 신청할 수 없습니다.

기존 수강 반과 시간이 겹치는 경우:

[E_STUDENT_CONFLICT] 기존 수강 반과 수업 시간이 겹칩니다.

이미 수강 중인 반인 경우:

[E_ALREADY_ENROLLED] 이미 수강 중인 반입니다.

수강 신청 조건을 만족하지 못하는 경우 해당 사유를 안내한다.

검사 순서	실패 조건	처리
1	반 ID 형식 오류	형식 오류를 출력하고 재입력
2	존재하지 않는 반	대상 없음 오류를 출력하고 재입력
3	학생이 퇴원 상태	신청 거부 후 학생 메뉴 복귀

4	반이 폐강 상태	신청 거부 후 반 ID 재입력
5	시작 날짜 당일 또는 이후	신청 기간 종료 안내 후 재입력
6	동일 반의 수강중 등록 존재	중복 신청 안내 후 재입력
7	정원 도달 또는 초과	정원 오류 안내 후 재입력
8	기존 수강과 일정 충돌	충돌 반을 안내하고 재입력
9	저장 실패	실패 안내 후 학생 메뉴 복귀

모든 검사를 통과한 경우에만 신청 확인을 받는다. 저장이 완료된 경우에만 신청 완료 메시지를 출력한다. 오류 코드는 공통 오류 코드 정의와 동일하게 사용한다.

6.4.5 내 수강 목록 조회

학생 본인이 현재 수강 중인 반을 조회한다.

```

===== 학생 메뉴 ({이름}님) > 내 수강 목록 조회 =====

반 ID | 반 이름 | 과목명 | 담당 강사 | 수업 요일 | 교시 | 수업 기간 | 수강 상태
-----
C0001 | A반 | 수학 | 김강사 | 월/수 | 3교시 | 2026-09-01~2026-12-20 | 수강중
C0003 | A반 | 영어 | 이강사 | 화/목 | 2교시 | 2026-09-01~2026-12-20 | 수강중

0. 이전 메뉴
번호를 입력하세요: 

```

현재 수강 중인 반이 없는 경우:

```

현재 수강 중인 반이 없습니다.

```

6.4.6 수강 취소

```

===== 학생 메뉴 ({이름}님) > 수강 취소 =====

수강 취소할 반 ID를 입력하세요: 

해당 반이 현재 수강 중인 경우 반 정보를 확인한 후 취소 여부를 확인한다.

반 ID: {반ID}

```

반 이름: {반이름}
과목명: {과목명}
담당 강사: {강사이름}

수강 취소하시겠습니까? (Y/N, 기본값 N):

Y를 입력하면

수강 취소가 완료되었습니다.

N 또는 Enter를 입력하면

작업을 취소했습니다. 변경된 내용이 없습니다.

수강 중이지 않은 반을 입력한 경우

[E_REF_MISSING] 해당 수강 정보를 찾을 수 없습니다.

6.4.7 로그아웃

로그아웃되었습니다.

===== 학원 운영 관리 프로그램 =====
사용자 ID:

6.5 강사 메뉴

강사는 다음 기능을 사용할 수 있다.

===== 강사 메뉴 ({이름}님) =====

- 1. 담당 반 조회
 - 2. 담당 반 수강생 조회
 - 3. 수업 일정 조회
 - 9. 로그아웃
 - 0. 프로그램 종료
- 번호를 입력하세요: ▢

6.5.1 담당 반 조회

강사는 본인이 담당하는 반의 목록을 조회할 수 있다.

===== 강사 메뉴 ({이름}님) > 담당 반 조회 =====

반 ID | 반 이름 | 과목명 | 수업 요일 | 교시 | 반 상태 | 현재 인원/정원

C0001 | A반 | 수학 | 월/수 | 3교시 | 모집중 | 8/10
C0003 | A반 | 영어 | 화/목 | 2교시 | 진행중 | 7/10

0. 이전 메뉴
번호를 입력하세요: ▢

강사는 본인이 담당하는 반의 정보만 조회할 수 있다.

6.5.2 담당 반 수강생 조회

담당 반 수강생 조회를 선택하면 로그인한 강사가 담당하는 반 목록을 먼저 출력한다.

===== 강사 메뉴 ({이름}님) > 담당 반 수강생 조회 =====

반 ID | 반 이름 | 과목명 | 요일 | 교시

C0001 | A반 | 수학 | 월/수 | 3~4교시
C0003 | A반 | 영어 | 화/목 | 2교시

조회할 반 ID를 입력하세요 (0: 이전 메뉴): ▢

로그인한 강사가 담당하는 반이 하나도 없는 경우에는 “담당하고 있는 반이 없습니다.” 메시지를 출력한 뒤 강사 메뉴로 돌아간다.

강사가 담당하는 반이 하나 이상인 경우에는 조회할 반 ID를 입력받는다. 입력한 반 ID가 로그인한 강사가 담당하는 반인 경우 해당 반의 현재 수강생 목록을 출력한다.

===== 강사 메뉴 ({이름}님) > 담당 반 수강생 조회 =====

반 ID: {반ID}
반 이름: {반이름}
과목명: {과목명}

학생 ID | 학생 이름 | 수강 상태

S0001 | 김민수 | 수강중
S0002 | 이지훈 | 수강중
S0005 | 이학생 | 수강중

수강생 목록을 출력한 뒤에는 담당 반 목록과 반 ID 입력 프롬프트를 다시 출력한다. 사용자는 다른 담당 반을 계속 조회하거나 0을 입력하여 강사 메뉴로 돌아갈 수 있다.

현재 수강 중인 학생이 없는 경우:

현재 수강 중인 학생이 없습니다.

본인이 담당하지 않는 반의 ID를 입력한 경우에는 해당 의 수강생 정보를 조회할 수 없다.

[E_NO_PERMISSION] 해당 반의 수강생 정보를 조회할 권한이 없습니다.

존재하지 않는 반 ID를 입력한 경우

[E_REF_MISSING] 해당 반을 찾을 수 없습니다.
반 ID를 다시 입력하세요: ▯

입력 처리 규칙은 다음과 같다.

- 0 입력 → 담당 반 수강생 조회를 종료하고 강사 메뉴로 돌아간다.
- 존재하지 않는 반 ID 입력 → “[E_REF_MISSING] 해당 반을 찾을 수 없습니다.”를 출력하고 반 ID를 다시 입력받는다.
- 존재하지만 본인이 담당하지 않는 반 ID 입력 → “[E_NO_PERMISSION] 해당 반의 수강생 정보를 조회할 권한이 없습니다.”를 출력하고 반 ID를 다시 입력받는다.
- 반 ID의 문법 형식이 올바르지 않은 경우 → 형식 오류 메시지를 출력하고 반 ID를 다시 입력받는다.

- 빈 입력 → 취소로 처리하여 강사 메뉴로 돌아간다.

6.5.3 수업 일정 조회

강사는 본인이 담당하는 반 중 가상 현재 일시의 날짜를 기준으로 현재 수업 기간에 포함되는 개설 상태의 반에 대한 수업 일정을 조회할 수 있다.

조회 대상은 다음 조건을 모두 만족하는 반으로 한다.

- 로그인한 강사가 담당 강사로 지정되어 있다.
- 반 상태가 개설이다.
- 시작 날짜 ≤ 가상 현재 일시의 날짜 ≤ 종료 날짜를 만족한다.

조회 대상 반의 수업 일정을 요일, 교시 순으로 정렬하여 출력한다.

===== 강사 메뉴 ({이름}님) > 수업 일정 조회 =====

요일 | 교시 | 반 ID | 반 이름 | 과목명

 월 | 3교시 | C0001 | A반 | 수학
 화 | 2교시 | C0003 | A반 | 영어
 수 | 3교시 | C0001 | A반 | 수학
 목 | 2교시 | C0003 | A반 | 영어

0. 이전 메뉴

번호를 입력하세요: □

조건을 만족하는 반이 하나도 없는 경우 “현재 진행 중인 담당 수업이 없습니다.”를 출력한다.

6.5.4 로그아웃

로그아웃되었습니다.

===== 학원 운영 관리 프로그램 =====

사용자 ID: □

6.6 원장 메뉴

로그인한 계정의 역할이 원장이면 다음 메뉴를 출력한다. 번호 입력 규칙과 오류 처리는 6.1.1, 6.1.2절, 확인 프롬프트는 6.1.3절을 따른다.

```
===== 원장 메뉴 =====
```

- 1. 학생 관리
 - 2. 강사 관리
 - 3. 과목 관리
 - 4. 반 관리
 - 5. 수강 등록 조회·취소
 - 6. 가상 현재 일시 변경
 - 7. 무결성 검사
 - 8. 비밀번호 관리
 - 9. 로그아웃
 - 0. 프로그램 종료
- 번호를 입력하세요: █

6.6.1 학생 관리

```
===== 원장 메뉴 > 학생 관리 =====
```

- 1. 목록 조회
 - 2. 상세 조회
 - 3. 검색
 - 4. 등록
 - 5. 수정
 - 6. 삭제
 - 0. 이전 메뉴
- 번호를 입력하세요: █

6.6.1.1 목록 조회

students.txt의 모든 학생 레코드를 학생 ID 오름차순으로 출력한다.

학생 목록에는 재원·퇴원 상태의 학생을 모두 포함한다.

표시 필드: 학생 ID, 이름, 연락처, 사용자 ID, 학생 상태, 현재 수강 반 수.

출력 예시:

학생ID	이름	연락처	사용자ID	상태	수강반수
------	----	-----	-------	----	------

S0001	김민수	010-1234-5678	std01	재원	2
S0002	이지훈	010-2345-6789	std02	재원	1

학생 없음 -> 등록된 학생이 없습니다.

6.6.1.2 상세 조회

부 프롬프트: 학생 ID를 입력하세요: []

출력 예시:

===== 학생 상세 (S0001) =====

학생ID: S0001

이름: 김민수

연락처: 010-1234-5678

사용자ID: std01

상태: 재원

수강 내역:

E0001	C0001	수학 A반	월 수	3~4교시	수강중	2026-09-01 15:30
-------	-------	-------	-----	-------	-----	------------------

E0004	C0004	영어 B반	화 목	5~6교시	수강중	2026-09-02 10:00
-------	-------	-------	-----	-------	-----	------------------

E0009	C0007	수학 C반	토	1~2교시	취소	2026-08-15 14:00
-------	-------	-------	---	-------	----	------------------

경우의 수:

존재하지 않는 학생 ID -> [E_REF_MISSING] 존재하지 않는 학생 ID입니다. 재입력

빈 입력 -> 취소

6.6.1.3 학생 검색

학생을 이름 또는 연락처로 검색한다.

===== 학생 검색 =====

1. 이름으로 검색

2. 연락처로 검색

3. 이전 메뉴

번호를 입력하세요: []

이름으로 검색하는 경우 검색어와 학생 이름의 합치 여부는 4.2.1절의 사람 이름 검색 규칙을 따른다. 연락처로 검색하는 경우 검색어와 학생 연락처의 합치 여부는 4.5절의 연락처 검색 규칙을 따른다.

검색 결과가 여러 개인 경우 합치되는 학생을 모두 출력하며, 검색 결과는 학생 ID 오름차순으로 출력한다.

검색 결과가 없으면 “검색 조건에 맞는 학생이 없습니다.”를 출력한다.

6.6.1.4 등록

부 프롬프트 순서: 이름 -> 연락처 -> 사용자 ID -> 비밀번호.

의미 규칙:

사용자 ID·비밀번호는 4.1.1·4.3 문법 형식을 만족해야 하고, 사용자 ID는 users.txt에 없는 새 ID여야 한다.

이름·연락처는 4.2.1·4.5 문법 형식을 만족해야 한다.

성공 시 자동 처리:

새 학생 ID(S + 4자리 숫자) 부여.

users.txt에 역할 학생 계정 추가.

students.txt에 상태 재원으로 레코드 추가.

확인 프롬프트: 학생 김민수(S0001)을(를) 등록하시겠습니까? (Y/N, 기본값 N): ☐

출력: [OK_STUDENT_ADD] 학생 S0001(김민수)이 등록되었습니다.

주요 오류 메시지:

형식 오류: [E_FILE_FORMAT] {필드명} 형식이 올바르지 않습니다.

사용자 ID 중복: [E_DUP_ID] 이미 존재하는 사용자 ID입니다.

저장 실패: [E_SAVE] 저장하지 못했습니다.

6.6.1.5 수정

퇴원 상태를 재원으로 변경하면 계정의 로그인을 다시 허용한다. 이전에 취소된 수강 등록은 자동으로 복원하지 않는다. 학생은 신청 조건을 만족하는 반에 새로 신청해야 한다.

부 프롬프트: 수정할 학생 ID를 입력하세요: ☐

수정 가능 필드: 이름, 연락처, 학생 상태. 수정 불가 필드: 학생 ID, 사용자 ID.

필드 선택 메뉴:

- 1. 이름
 - 2. 연락처
 - 3. 학생 상태 (재원 <-> 퇴원)
 - 0. 이전 메뉴

의미 규칙: 학생 상태를 퇴원으로 바꿀 때 그 학생의 수강중 상태 수강 등록이 하나라도 있으면 다음 경고를 출력하고 원장 확인을 받는다.

[W_STUDENT_HAS_ENROLLMENTS] 이 학생은 다음 반에 수강중 수강 등록되어 있습니다:
C0001 수학 A반
C0004 영어 B반
퇴원 처리 시 위 수강 등록은 모두 자동으로 취소 처리됩니다.
계속하시겠습니까? (Y/N, 기본값 N): ☐

성공 시 자동 처리 (퇴원 처리 시):

students.txt의 학생 상태를 퇴원으로 변경.

그 학생의 수강중 상태 수강 등록을 모두 취소로 변경, 취소 일시 = 가상 현재 일시.

출력: [OK_STUDENT_UPDATE] 학생 S0001의 정보가 수정되었습니다. (퇴원 처리 시) 수강중 수강 등록 2건이 함께 취소되었습니다. 추가.

6.6.1.6 삭제

부 프롬프트: 삭제할 학생 ID를 입력하세요: []

의미 규칙:

그 학생의 수강 등록 레코드(수강중·취소 모두)가 하나라도 있으면 삭제할 수 없다.

삭제 시 students.txt와 users.txt에서 해당 학생·계정 레코드를 함께 제거한다.

경우의 수:

수강 등록 있음:

[E_STUDENT_HAS_HISTORY] 이 학생에게는 수강 등록 이력이 있어 삭제할 수 없습니다. 기존 기록을 유지하려면 학생 수정 메뉴에서 상태를 '퇴원'으로 변경하세요.

수강 등록 없음 -> 확인 프롬프트 -> 삭제.

확인 프롬프트: 학생 S0005(박서연)을(를) 삭제하시겠습니까? (Y/N, 기본값 N): []

출력: [OK_STUDENT_DELETE] 학생 S0005이 삭제되었습니다.

6.6.2 강사 관리

```
===== 원장 메뉴 > 강사 관리 =====
1. 목록 조회
2. 상세 조회
3. 검색
4. 등록
5. 수정
6. 삭제
0. 이전 메뉴
번호를 입력하세요: []
```

6.6.2.1 목록 조회

teachers.txt의 모든 강사 레코드를 강사 ID 오름차순으로 출력한다.

표시 필드: 강사 ID, 이름, 연락처, 사용자 ID, 강사 상태, 담당 반 수.

출력 예시:

강사ID	이름	연락처	사용자ID	상태	담당반수
T0001	이준호	010-2345-6789	teacher1	재직	2
T0002	박은지	010-3456-7890	teacher2	재직	1
T0003	최도현	010-4567-8901	teacher3	퇴직	0

강사 없음 -> 등록된 강사가 없습니다.

6.6.2.2 상세 조회

부 프롬프트: 강사 ID를 입력하세요: []

출력 예시:

```
===== 강사 상세 (T0001) =====
강사ID: T0001
이름: 이준호
연락처: 010-2345-6789
사용자ID: teacher1
상태: 재직
담당 반:
  C0001 수학 A반 월 수 3~4교시 개설
  C0004 수학 B반 화 목 5~6교시 개설
```

경우의 수:

존재하지 않는 강사 ID -> [E_REF_MISSING] 존재하지 않는 강사 ID입니다. 재입력

빈 입력 -> 취소

6.6.2.3 강사 검색

강사를 이름 또는 연락처로 검색한다.

```
===== 강사 검색 =====
1. 이름으로 검색
2. 연락처로 검색
```

3. 이전 메뉴 번호를 입력하세요: ▢

이름으로 검색하는 경우 검색어와 학생 이름의 합치 여부는 4.2.1절의 사람 이름 검색 규칙을 따른다. 연락처로 검색하는 경우 검색어와 학생 연락처의 합치 여부는 4.5절의 연락처 검색 규칙을 따른다.

검색 결과가 여러 개인 경우 합치되는 강사를 모두 출력하며, 검색 결과는 강사 ID 오름차순으로 출력한다.

검색 결과가 없으면 “검색 조건에 맞는 강사가 없습니다.”를 출력한다.

6.6.2.4 등록

부 프롬프트 순서: 이름 -> 연락처 -> 사용자 ID -> 비밀번호.

의미 규칙: 사용자 ID·비밀번호·이름·연락처가 4장의 문법 형식을 만족해야 하며, 사용자 ID는 users.txt에 없는 새 ID여야 한다.

성공 시 자동 처리: 새 강사 ID(T + 4자리 숫자) 부여, users.txt에 역할 강사 계정 추가, teachers.txt에 상태 재직으로 레코드 추가.

확인 프롬프트: 강사 이준호(T0001)을(를) 등록하시겠습니까? (Y/N, 기본값 N): ▢

출력: [OK_TEACHER_ADD] 강사 T0001(이준호)이 등록되었습니다.

주요 오류 메시지:

형식 오류: [E_FILE_FORMAT] {필드명} 형식이 올바르지 않습니다.

사용자 ID 중복: [E_DUP_ID] 이미 존재하는 사용자 ID입니다.

저장 실패: [E_SAVE] 저장하지 못했습니다.

6.6.2.5 수정

퇴직 상태를 재직으로 변경하면 계정의 로그인을 다시 허용하고 신규 반 배정 대상에 포함한다. 기존 반의 담당 강사 정보는 상태 복귀만으로 변경하지 않는다.

부 프롬프트: 수정할 강사 ID를 입력하세요: ▯

수정 가능 필드: 이름, 연락처, 강사 상태. 수정 불가 필드: 강사 ID, 사용자 ID.

필드 선택 메뉴:

- 1. 이름
- 2. 연락처
- 3. 강사 상태 (재직 <-> 퇴직)
- 0. 이전 메뉴

의미 규칙: 강사 상태를 퇴직으로 바꿀 때 그 강사가 담당 중인 개설 상태 반이 있으면 다음 경고를 출력하고 원장 확인을 받는다.

[W_TEACHER_HAS_CLASSES] 이 강사는 다음 반을 담당 중입니다:
C0001 수학 A반
C0004 수학 B반
퇴직 처리 후에도 기존 반 정보는 유지되지만, 새 반 개설·수정 시 배정 대상에서 제외됩니다.
계속하시겠습니까? (Y/N, 기본값 N): ▯

출력: [OK_TEACHER_UPDATE] 강사 T0001의 정보가 수정되었습니다.

6.6.2.6 삭제

부 프롬프트: 삭제할 강사 ID를 입력하세요: ▯

의미 규칙: 그 강사를 담당 강사로 참조하는 반이 하나라도 있으면 삭제할 수 없다.

경우의 수:

참조 반 있음:

[E_TEACHER_IN_USE] 이 강사를 참조하는 반이 있어 삭제할 수 없습니다.
C0001 수학 A반
C0004 수학 B반
강사 수정 메뉴에서 상태를 '퇴직'으로 변경하세요.

참조 반 없음 -> 확인 프롬프트 -> 삭제 (teachers.txt와 users.txt에서 함께 제거).

확인 프롬프트: 강사 T0003(최도현)을(를) 삭제하시겠습니까? (Y/N, 기본값 N): ▯

출력: [OK_TEACHER_DELETE] 강사 T0003이 삭제되었습니다.

6.6.3 과목 관리

```
===== 원장 메뉴 > 과목 관리 =====  
1. 목록 조회  
2. 상세 조회  
3. 과목 검색  
4. 등록  
5. 수정  
6. 삭제  
0. 이전 메뉴  
번호를 입력하세요: █
```

6.6.3.1 목록 조회

subjects.txt의 모든 과목을 과목 ID 오름차순으로 출력한다.

표시 필드: 과목 ID, 과목명, 개설된 반 수.

출력 예시:

과목ID	과목명	개설반수
P0001	수학	3
P0002	중등 수학	1
P0003	영어	2

과목 없음 -> 등록된 과목이 없습니다.

6.6.3.2 상세 조회

부 프롬프트: 과목 ID를 입력하세요: █

해당 과목의 이름과 그 과목으로 개설된 반 목록(반 ID, 반 이름, 담당 강사, 요일, 교시)을 출력한다.

출력 예시:

```
===== 과목 상세 (P0001) =====  
과목ID: P0001  
과목명: 수학  
개설된 반:  
C0001 수학 A반   T0001 이준호   월 수 3~4교시  
C0002 수학 B반   T0002 박은지   화 목 5~6교시
```

6.6.3.3 과목 검색

과목명을 입력받아 과목을 검색한다.

부 프롬프트: 검색할 과목명을 입력하세요: ▢

검색어와 과목명의 합치 여부는 4.2.2절의 과목명 검색 규칙을 따른다.

검색 조건에 합치되는 과목이 여러 개인 경우 합치되는 모든 과목을 과목 ID 오름차순으로 출력한다.

검색 조건에 합치되는 과목이 없는 경우 검색 조건에 맞는 과목이 없습니다.를 출력한다.

6.6.3.4 등록

부 프롬프트: 과목명: ▢

의미 규칙: 과목명은 4.2.2 문법 형식을 만족해야 한다. 과목명 자체의 중복은 허용한다.

성공 시 자동 처리: 새 과목 ID(P + 4자리 숫자) 부여.

확인 프롬프트: 과목 "수학"(P0001)을(를) 등록하시겠습니까? (Y/N, 기본값 N): ▢

출력: [OK_SUBJECT_ADD] 과목 P0001(수학)이 등록되었습니다.

6.6.3.5 수정

부 프롬프트: 수정할 과목 ID를 입력하세요: ▢ -> 새 과목명 입력.

수정 가능 필드: 과목명만.

의미 규칙: 과목명 변경은 이미 개설된 반의 소속 과목명 표시에도 즉시 반영된다.

출력: [OK_SUBJECT_UPDATE] 과목 P0001의 이름이 수정되었습니다.

6.6.3.6 삭제

부 프롬프트: 삭제할 과목 ID를 입력하세요: ▢

의미 규칙: 그 과목을 참조하는 반이 하나라도 있으면 삭제할 수 없다.

경우의 수:

참조 반 있음:

[E_SUBJECT_IN_USE] 이 과목을 참조하는 반이 있어 삭제할 수 없습니다.
C0001 수학 A반
C0002 수학 B반

참조 반 없음 -> 확인 프롬프트 -> 삭제.

6.6.4 반 관리

```
===== 원장 메뉴 > 반 관리 =====
1. 목록 조회
2. 상세 조회
3. 개설
4. 수정
5. 폐강
0. 이전 메뉴
번호를 입력하세요: █
```

6.6.4.1 목록 조회

classes.txt의 모든 반을 반 ID 오름차순으로 출력한다.

표시 필드: 반 ID, 반 이름, 과목명, 담당 강사, 요일, 교시, 기간, 반 상태, 현재/정원.

출력 예시:

반ID	반이름	과목	강사	요일	교시	기간	상태	인원
C0001	A반	수학	이준호	월 수	3~4교시	2026-09-01 ~ 2026-12-20	개설	8/10
C0002	B반	수학	박은지	화 목	5~6교시	2026-09-01 ~ 2026-12-20	개설	5/12
C0005	A반	영어	최도현	토	1~4교시	2026-01-10 ~ 2026-02-28	폐강	0/15

6.6.4.2 상세 조회

부 프롬프트: 반 ID를 입력하세요: █

출력 예시:

```
===== 반 상세 (C0001) =====
반ID:   C0001
반이름: A반
과목:   P0001 수학
담당강사: T0001 이준호
요일:   월 수
교시:   3~4교시
기간:   2026-09-01 ~ 2026-12-20
정원:   10
현재인원: 8
상태:   개설

수강생:
```


S0001 김민수 수강중
S0007 이지훈 수강중
...

6.6.4.3 개설

부 프롬프트 순서:

과목 ID:
반 이름:
담당 강사 ID:
요일 목록 (예: 월 수):
시작 교시:
종료 교시:
시작 날짜 (YYYY-MM-DD):
종료 날짜 (YYYY-MM-DD):
정원:

의미 규칙:

- 과목 ID가 subjects.txt에 존재해야 한다.
- 담당 강사 ID가 teachers.txt에 존재하고 강사 상태가 재직이어야 한다.
- 요일 목록·교시·날짜·정원이 각 데이터 요소의 문법 형식을 만족해야 한다.
- 시작 교시 ≤ 종료 교시.
- 시작 날짜 ≤ 종료 날짜.
- 시작 날짜 ≥ 가상 현재 일시의 날짜.
- 99 ≥ 정원 ≥ 1.
- 강사 시간 충돌 검사: 새 반의 수업 일정이 그 강사의 다른 개설 상태 반과 4.12.1절의 일정 충돌 조건을 만족하지 않아야 한다.
- 반 이름이 같은 과목 안의 다른 반과 중복되지 않아야 한다.

최종 요약 및 확인:

과목: P0001 수학
반이름: A반
담당강사: T0001 이준호
요일: 월 수
교시: 3~4교시
기간: 2026-09-01 ~ 2026-12-20
정원: 10

강사 시간 충돌 검사: 이상 없음

반 C0001(수학 A반)을(를) 개설하시겠습니까? (Y/N, 기본값 N):

성공 시 자동 처리: 새 반 ID(C + 4자리 숫자) 부여, 반 상태 = 개설.

주요 오류 메시지:

- 존재하지 않는 과목 ID / 강사 ID: [E_REF_MISSING]
- 퇴직 강사 배정: [E_TEACHER_RETIRED] 퇴직 상태의 강사는 새 반에 배정할 수 없습니다.
- 교시 순서: [E_FILE_FORMAT] 종료 교시는 시작 교시와 같거나 이후여야 합니다.
- 날짜 순서: [E_FILE_FORMAT] 종료 날짜는 시작 날짜와 같거나 이후여야 합니다.
- 시작 날짜 과거: [E_DATE_PAST] 시작 날짜는 현재 날짜({가상현재날짜}) 이후여야 합니다.
- 정원 형식·범위 오류: [E_FILE_FORMAT] 정원은 1 이상 99 이하의 정수여야 합니다.
- 강사 시간 충돌:

[E_TEACHER_CONFLICT] 담당 강사의 다음 반과 수업 시간이 충돌합니다:
C0003 수학 C반 월 4~5교시 2026-09-01 ~ 2026-12-20

반 이름 중복: [E_CLASS_NAME_DUPLICATE] 같은 과목에 같은 이름의 반이 이미 있습니다.

6.6.4.4 수정

부 프롬프트: 수정할 반 ID를 입력하세요: □

진입 차단: 반 상태가 폐강 -> [E_CLASS_CLOSED] 폐강된 반은 수정할 수 없습니다.

수정 가능 필드: 반 이름, 담당 강사, 정원. 수정 불가 필드: 반 ID, 과목 ID, 요일 목록, 교시, 기간, 반 상태. (수업 일정을 바꾸려면 반을 폐강한 뒤 새로 개설한다.)

필드 선택 메뉴:

1. 반 이름
2. 담당 강사
3. 정원
0. 이전 메뉴

필드별 의미 규칙:

- 반 이름: 같은 과목 안 중복 금지.
- 담당 강사: 새 강사가 재직이어야 하고, 새 강사 기준으로 강사 시간 충돌 재검사(수정 대상 반 자신은 제외).
- 정원: $99 \geq \text{새 정원} \geq 1$ 이고, 새 정원 $\geq$ 현재 활성 수강 등록 수이어야 함.

정원 축소 오류: [E_CAPACITY_UNDER] 새 정원은 현재 수강중 학생 등록 수보다 작을 수 없습니다.

정원 '0' 입력 오류: 정원은 1 이상 99 이하의 정수여야 합니다.

성공 시 출력: [OK_CLASS_UPDATE] 반 C0001의 정보가 수정되었습니다.

6.6.4.5 폐강

부 프롬프트: 폐강할 반 ID를 입력하세요: []

진입 차단: 이미 폐강 상태 -> [E_CLASS_ALREADY_CLOSED] 이미 폐강된 반입니다.

경고 (수강생 존재 시):

[W_CLASS_HAS_STUDENTS] 이 반에는 현재 수강중 수강생 8명이 있습니다.
폐강 시 해당 학생들의 수강 등록은 자동으로 취소 처리됩니다.

확인 프롬프트: 반 C0001(수학 A반)을(를) 폐강하시겠습니까? (Y/N, 기본값 N): []

성공 시 자동 처리:

classes.txt의 반 상태를 폐강으로 변경.

그 반에 연결된 수강중 상태 수강 등록을 모두 취소로 변경, 취소 일시 = 가상 현재 일시.

출력: [OK_CLASS_CLOSE] 반 C0001이 폐강되었습니다. 수강중 수강 등록 8건이 함께 취소되었습니다.

6.6.5 수강 등록 조회·취소

===== 원장 메뉴 > 수강 등록 조회·취소 =====
1. 전체 수강 등록 조회
2. 반별 수강 등록 조회
3. 학생별 수강 등록 조회
4. 수강 등록 취소
0. 이전 메뉴
번호를 입력하세요: []

6.6.5.1 전체 수강 등록 조회

enrollments.txt의 모든 수강 등록 레코드를 등록 ID 오름차순으로 출력한다.

표시 필드: 등록 ID, 학생 ID, 학생 이름, 반 ID, 반 이름, 등록 일시, 등록 상태, 취소 일시.

출력 예시:

등록ID	학생ID	이름	반ID	반이름	등록일시	상태	취소일시
E0001	S0001	김민수	C0001	수학 A반	2026-08-27 15:30	수강중	-
E0002	S0002	이지훈	C0001	수학 A반	2026-08-28 14:00	수강중	-
E0003	S0003	박서연	C0002	영어 B반	2026-08-29 16:20	취소	2026-09-05 10:00

수강 등록 없음 -> 등록된 수강 등록이 없습니다.

6.6.5.2 반별 수강 등록 조회

부 프롬프트: 반 ID를 입력하세요: ▯

출력 예시:

```
===== 반별 수강 등록 (C0001 수학 A반) =====
등록ID  학생ID  이름      등록일시      상태  취소일시
E0001   S0001   김민수    2026-08-27 15:30  수강중  -
E0002   S0002   이지훈    2026-08-28 14:00  수강중  -
E0005   S0007   최도현    2026-08-29 11:20  취소    2026-09-07 09:00
수강중 인원: 2 / 정원: 10
```

경우의 수:

존재하지 않는 반 ID -> [E_REF_MISSING] 존재하지 않는 반 ID입니다. 재입력
등록 없음 -> 해당 반에 수강 등록이 없습니다.

6.6.5.3 학생별 수강 등록 조회

부 프롬프트: 학생 ID를 입력하세요: ▯

출력: 해당 학생의 수강 등록 목록. 등록 ID, 반 ID, 반 이름, 요일, 교시, 등록 일시, 등록 상태, 취소 일시.

경우의 수:

존재하지 않는 학생 ID -> [E_REF_MISSING] 존재하지 않는 학생 ID입니다. 재입력
등록 없음 -> 해당 학생의 수강 등록이 없습니다.

6.6.5.4 수강 등록 취소

원장은 학생 본인의 수강 취소와 별개로, 임의의 수강 등록을 취소할 수 있다.

부 프롬프트: 취소할 수강 등록 ID를 입력하세요: ▯

진입 차단:

이미 취소 상태 -> [E_ENROLL_ALREADY_CANCELLED] 이미 취소된 수강 등록입니다.

존재하지 않는 등록 ID -> [E_REF_MISSING] 존재하지 않는 수강 등록 ID입니다.

대상 정보 출력:

```
===== 취소 대상 확인 =====
등록ID:  E0001
학생:    S0001 김민수
반:      C0001 수학 A반
등록일시: 2026-09-01 15:30
상태:    수강중
```

확인 프롬프트: 이 수강 등록을 취소하시겠습니까? (Y/N, 기본값 N):

성공 시 자동 처리:

enrollments.txt의 해당 레코드의 상태를 취소로 변경.

취소 일시 = 가상 현재 일시.

출력: [OK_ENROLL_CANCEL] 수강 등록 E0001이 취소되었습니다.

6.6.6 무결성 검사

원장이 이 항목을 선택하면 5.4절의 검사 단계 표에 따라 모든 데이터 파일의 문법·참조·의미 규칙을 순서대로 재검사한다. 프로그램 시작 시의 무결성 검사를 통과했다라도 실행 중에 사용자가 데이터 파일을 외부에서 편집했을 수 있으므로, 원장은 언제든지 재검사를 수행할 수 있다. 재검사에서 오류가 발견되면 오류 내용을 출력하고, 파일을 고친 뒤 프로그램을 다시 실행하도록 안내한다.

출력 예시 (이상 없음):

```
===== 무결성 검사 =====
[1/8] 파일 존재·읽기 가능 확인 ..... OK
[2/8] UTF-8 디코딩 확인 ..... OK
[3/8] 행의 필드 수·문법 검사 ..... OK
[4/8] 필드별 형식·범위 검사 ..... OK
[5/8] 파일 내 ID 중복 검사 ..... OK
[6/8] 파일 간 참조 무결성 검사 ..... OK
[7/8] 수업 시간·정원·등록 관계 검사 ..... OK
[8/8] 가상 시각 역행 검사 ..... OK

[OK_INTEGRITY] 무결성 검사 결과: 이상 없음.
```

출력 예시 (오류 발견):

===== 무결성 검사 =====

[1/8] 파일 존재·읽기 가능 확인 OK
[2/8] UTF-8 디코딩 확인 OK
[3/8] 행의 필드 수·문법 검사 FAIL
 [E_FILE_FORMAT] classes.txt: 15 필드 개수 부족
[4/8] 필드별 형식·범위 검사 (classes.txt 제외) . OK
[5/8] 파일 내 ID 중복 검사 (classes.txt 제외) .. OK
[6/8] 파일 간 참조 무결성 검사 (classes.txt 제외) ... OK
[7/8] 수업 시간·정원·등록 관계 검사 (classes.txt 제외) ... OK
[8/8] 가상 시각 역행 검사 FAIL

 [E_TIME_BACKWARD] datetime.txt(2026-09-27 15:30)가 enrollments.txt의
 최근 등록 일시(2026-09-30 10:00)보다 이릅니다.

===== 검사 결과 요약 =====

오류 2건 (파일 문법 1, 의미 규칙 1)

데이터 파일에 오류가 있어 프로그램을 종료합니다. 파일을 수정한 뒤 다시 실행하세요.

발견된 오류는 화면에 나열만 하고, 데이터 파일은 자동으로 수정하지 않는다.

이상이 없으면 원장 메뉴로 복귀한다. 오류가 하나라도 발견되면 5.4절에 따라 프로그램을 종료하며, 사용자가 데이터 파일을 복구한 뒤 다시 실행해야 한다.

출력 후: 원장 메뉴로 복귀.

6.6.7 로그아웃

로그아웃되었습니다.

===== 학원 운영 관리 프로그램 =====

사용자 ID: □

6.6.8 계정 비밀번호 관리

===== 원장 메뉴 > 계정 비밀번호 관리 =====

1. 학생·강사 비밀번호 초기화

2. 원장 비밀번호 변경

0. 이전 메뉴

번호를 입력하세요: □

번호 입력 규칙과 오류 처리는 6.1.1, 6.1.2절을 따른다.

학생·강사 비밀번호는 원장이 초기화한다. 원장은 초기화할 사용자 ID를 입력한다.

프로그램은 해당 계정이 존재하고 역할이 학생 또는 강사인지를 확인한다. 대상 학생·강사 정보를 표시한 뒤 새 비밀번호를 입력받는다.

부 프롬프트: 초기화할 사용자 ID를 입력하세요: ▯

진입 차단:

존재하지 않는 사용자 ID

-> [E_REF_MISSING] 존재하지 않는 사용자 ID입니다.

역할이 원장인 계정

-> [E_ROLE_INVALID] 학생 또는 강사 계정만 초기화할 수 있습니다.

대상 계정이 확인되면 해당 사용자 정보를 출력한다.

새 비밀번호는 4.3절의 형식을 만족해야 하며, 확인을 위해 같은 비밀번호를 한 번 더 입력받는다. 두 값이 일치하면 최종 확인을 받는다. 동의한 경우 users.txt의 해당 비밀번호만 변경한다. 비밀번호는 화면과 결과 메시지에 출력하지 않는다. 저장 성공 후 초기화 완료 메시지를 출력한다.

원장 본인의 변경은 다음처럼 구분한다.

원장 계정의 비밀번호 변경은 현재 비밀번호 확인 후 새 비밀번호를 두 번 입력받아 처리한다. 원장이 현재 비밀번호를 기억하지 못하는 경우에는 프로그램 내부에서 비밀번호를 초기화하는 기능을 제공하지 않는다. 프로그램을 종료한 상태에서 users.txt의 원장 계정 비밀번호 필드를 직접 수정하여 복구한다. 직접 수정한 비밀번호 역시 4.3절의 문법 형식을 만족해야 하며, 프로그램을 다시 실행할 때 5.4절의 무결성 검사를 통과해야 한다.

6.7 종료

6.7.1 정상 종료 — 메뉴에서 선택

역할별 최상위 메뉴에서 0을 입력한 경우다. 6.1.4절과 동일하게 확인을 받는다.

프로그램을 종료하시겠습니까? (Y/N, 기본값 N): ▯

Y 입력 시는 다음과 같이 처리한다 :

프로그램을 종료합니다.

N(또는 Enter) 입력 시 확인 프롬프트 취소 문구(6.1.3절)를 출력하고 메뉴로 복귀한다. 로그인 전 화면의 사용자 ID 입력 단계에서 0을 입력하면, 사용자 ID의 문법 형식 검사보다 우선하여 종료 명령으로 처리하고 동일한 종료 확인을 받는다.

6.7.2 로그인 연속 실패로 인한 종료

6.2.3절과 동일하다. 한 번의 실행 중 인증 실패가 연속 5회 발생하면 확인 없이 종료한다.

[E_AUTH_FAIL] 로그인 정보가 올바르지 않습니다.
로그인 시도 횟수를 초과했습니다. 프로그램을 종료합니다.

이 경로는 사용자의 명시적 종료 의사가 아니므로 Y/N 확인을 두지 않는다. 데이터 파일은 변경하지 않는다.

6.7.3 입력 도중 중단 (Ctrl+C, EOF)

공통 입력 처리 규칙에 따라, 어떤 입력 단계에서든 Ctrl+C 또는 입력 종료 신호(EOF, 예: Ctrl+D)가 발생하면 진행 중이던 입력과 그 작업을 즉시 취소하고 다음 메시지와 함께 종료한다.

입력이 중단되었습니다. 저장되지 않은 변경 사항 없이 프로그램을 종료합니다.

이 종료는 Y/N 확인을 거치지 않는다. 신호 자체가 "더 이상 입력을 계속할 수 없음"을 뜻하므로 별도 확인을 시도하지 않는다.

저장 절차가 이미 시작되어 임시 파일까지 기록된 상태에서 신호가 발생한 경우, 단일 파일을 변경하는 작업에서는 `os.replace()` 교체가 끝나기 전까지 원본이 변경되지 않으므로 원본은 항상 이전 상태로 유지된다. 여러 파일을 변경하는 작업은 5.4절의 저장 절차를 따른다. 남은 임시 파일은 다음 실행 시작 시 정리한다.

6.7.4 시작 시 치명적 오류로 인한 종료

5.4절의 무결성 검사(①~⑧) 중 어느 단계에서든 오류가 발견되면 로그인 화면으로 넘어가지 않고 즉시 종료한다.

[E_FILE_FORMAT] students.txt 3행: 필드 수가 5개가 아니라 4개입니다.
필수 데이터 파일에 오류가 있어 프로그램을 시작할 수 없습니다. 파일을 수정한 뒤 다시 실행하세요.

data 디렉토리는 존재하지만 필수 데이터 파일 중 일부가 없거나 읽을 수 없는 경우, 인코딩 오류인 경우도 각각 `E_FILE_MISSING`, `E_FILE_READ`, `E_FILE_FORMAT`으로 동일하게 처리하고 종료한다. 다만 **data** 디렉토리 자체가 존재하지 않는 경우는 최초 실행으로 보고 2.3.1절의 절차를 수행하며, 오류로 처리하지 않는다.

여러 파일에 걸쳐 오류가 있는 경우, 발견되는 오류를 모두 모아 한 번에 출력한 뒤 종료한다(하나만 고치고 재실행했다가 다음 오류를 또 만나는 상황을 줄이기 위함). 다만 어떤 파일의 문법 오류로 인해 검사할 수 없게 된 뒤 단계의 참조 오류는 함께 출력하지 않는다.

6.7.5 종료 시 공통 사항

항목	처리
종료 코드	정상 종료(6.7.1)는 성공 종료로, 그 외(6.7.2~6.7.4)는 실패 종료로 운영체제에 반환한다.
화면 출력 위치	모든 종료 메시지는 표준 출력에 출력한다. 비밀번호 등 민감 정보는 어떤 종료 메시지에도 포함하지 않는다.
로그아웃과의 구분	6.2.5절의 로그아웃은 종료가 아니라 로그인 화면으로 돌아가는 것이며, 이 절이 다루는 종료와는 별개다.
재실행 시 상태	어떤 종료 경로로 끝났든, 다음 실행은 항상 6.2.1절의 로그인 화면부터 다시 시작한다. 직전 세션의 로그인 상태나 메뉴 위치는 저장하지 않는다.

7. 기타 처리

7.1 구현 완료 조건

프로그램의 각 기능은 정상적인 입력뿐만 아니라 주요 오류 및 예외 상황에서도 요구된 동작을 수행하고, 처리 결과가 데이터 파일에 정확하게 반영되는 경우 구현이 완료된 것으로 판단한다.

7.1.1 기능 테스트 기준

테스트 항목	테스트 상황	기대 결과
충돌 검사	동일한 시간에 학생이 이미 수강 중인 반에 중복으로 수강 신청	수강 신청을 거부하고 충돌 오류를 표시하며 기존 데이터는 변경되지 않는다.

강사 일정 충돌	동일한 시간에 이미 담당 중인 수업과 겹치는 반을 등록	등록을 거부하고 충돌 오류를 표시하며 기존 데이터는 유지된다.
취소 후 재신청	기존 수강 신청을 취소한 후 동일한 반에 다시 수강 신청	기존 수강 기록은 취소 상태로 유지하고, 새로운 수강 등록 식별자를 생성하여 새로운 수강 신청을 저장한다.
학생 퇴원 연쇄 처리	학생의 상태를 퇴원으로 변경	해당 학생의 수강중인 모든 수강 등록을 취소로 변경하고 취소 일시를 기록한다.
반 폐강 연쇄 처리	개설된 반을 폐강 상태로 변경	해당 반의 수강중인 모든 수강 등록을 취소로 변경하고 취소 일시를 기록한다.
재실행 후 저장 유지	데이터를 변경하고 프로그램을 종료한 후 다시 실행	종료 전에 저장한 변경 내용이 그대로 유지되며 중복 레코드가 생성되지 않는다.
부분 저장 실패	하나의 기능에서 여러 관련 데이터 파일을 저장하는 중 일부 파일의 저장에 실패	해당 작업 전체를 실패 처리하고 일부 데이터만 변경된 상태가 남지 않도록 기존 상태를 유지한다. 성공 메시지를 출력하지 않는다.
무결성 검사	데이터 파일의 문법 또는 참조 관계에 오류가 있는 상태에서 무결성 검사 실행	오류를 탐지하고 정의된 오류 처리 규칙에 따라 오류 코드 또는 메시지를 출력한다.

7.1.2 완료 판단 기준

각 기능은 다음 조건을 모두 만족하는 경우 구현 완료로 판단한다.

1. 정상 입력에 대해 요구된 기능이 정상적으로 수행된다.
2. 정의된 오류 및 예외 상황을 탐지하고 적절한 오류 메시지를 출력한다.
3. 오류가 발생한 경우 기존의 정상 데이터가 임의로 변경되지 않는다.
4. 수강 취소, 학생 퇴원, 반 폐강 등 연쇄 변경이 관련 데이터에 일관되게 반영된다.
5. 프로그램을 종료한 후 다시 실행해도 저장된 데이터가 정상적으로 유지된다.
6. 여러 데이터 파일을 함께 변경하는 작업에서 부분 저장이 발생하지 않는다.
7. 테스트 시나리오의 기대 결과와 실제 결과가 일치한다.